

MID-SEMESTER STUDY SERIES • GRAPH THEORY • DAY 1

CS-5136: Graph Theory

Day 1 — From Real Problems to Graphs: the Shortest Superstring Problem

pages 1–33 • motivation, the overlap function, and the Hamiltonian-path reduction — with full derivations and worked examples

Switching source — and what that changes

- This deck now runs on your actual course material — CS-5136: Graph Theory (A. Bhattacharjee, NIT Silchar, 230 slide-pages) — since that's what your mid-sem really covers.
- The “Day” count restarts here at Day 1 for this book. The earlier Combinatorial Optimization deck is a different course — keep it as background reading, not mid-sem syllabus.
- Where it genuinely helps — today's TSP/Hamiltonian-path connection, spanning trees later on — I'll point to the exact spot in the Held Combinatorial Optimization notes as a cross-reference, since you already have that material.

SAME PROMISE AS BEFORE: every sketchy or ambiguous step gets a full, correct derivation; every dense idea gets a worked example; you get practice questions with answers at the end.

What Day 1 covers

In today's deck

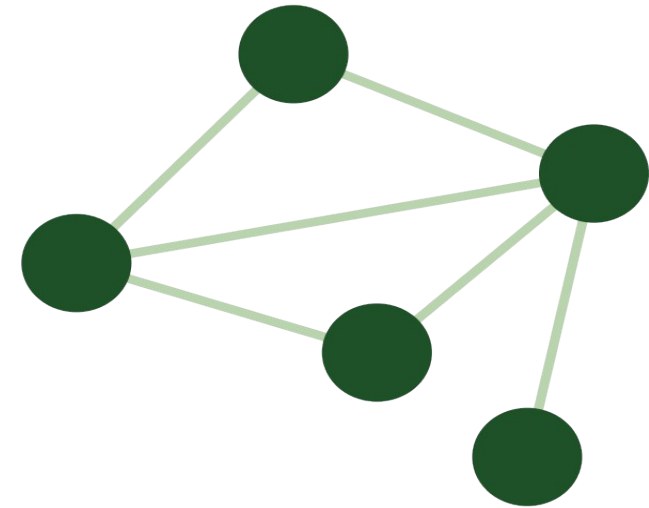
- The real-world problems that motivate graph theory (railway planning, DNA sequencing)
- The Shortest Superstring Problem (SSP) — formal definition
- The overlap function — definition, worked example, and a key subtlety (it's directional)
- The reduction of SSP to a shortest-Hamiltonian-path problem — built and proved correctly
- A fully worked 5-fragment example, end to end
- 5 practice questions with full answers

A note on “pages”

- Today = slide-pages 1–33 of 230 — the “Introduction” chapter, right up to where formal graph definitions begin (page 34).
- Many raw pages are just build/reveal steps of one idea (one animation spread across up to 9 pages) — I've compressed those into single explanatory slides.
- So this deck's ~19 slides carry the full content of those 33 pages, just without the repeats.

Why model anything as a graph?

- A graph is just: a set of items, plus a set of connections between pairs of them.
- Telephone / computer networks
- Transportation networks (bus, subway, train, plane)
- Social networks (who is friends with whom)
- Family trees, evolutionary trees
- Molecular graphs (atoms and chemical bonds)
- Many data structures in Computer Science itself



items + connections = a graph

Delhi Metro: three planning problems

NEW TIMETABLE

The schedule repeats every 60 minutes.
Each departure/arrival gets a number mod 60.

What must these numbers satisfy?

Previews: algebraic / cycle-space view of graphs (~pg. 60)

STOCK CIRCULATION

Trains run a repeating weekly cycle. What's the minimum number of train-units needed to cover it?

A classic circulation/flow problem — exactly your Comb. Opt. reference book's territory

CIRCULAR ROUTE

A circular line where every segment between stations must be traversed exactly once.

This is precisely an Eulerian circuit — formal treatment ~pg. 137

Assembling a Genome from Fragments

- Sequencing methods (Sanger 1977 / Gilbert 1977) read only short fragments of DNA — about 500–700 nucleotides at a time — not the whole genome at once.
- So the genome is sheared into millions of small, overlapping fragments (“reads”) first, then each one is read separately.
- The computational challenge: reassemble the short reads back into one full genomic sequence — a “superstring” containing all of them.
- Until the late 1990s this “shotgun fragment assembly” was viewed as computationally intractable for genome-sized inputs — exactly the kind of problem graph theory turns out to help with.

The Shortest Superstring Problem (SSP)

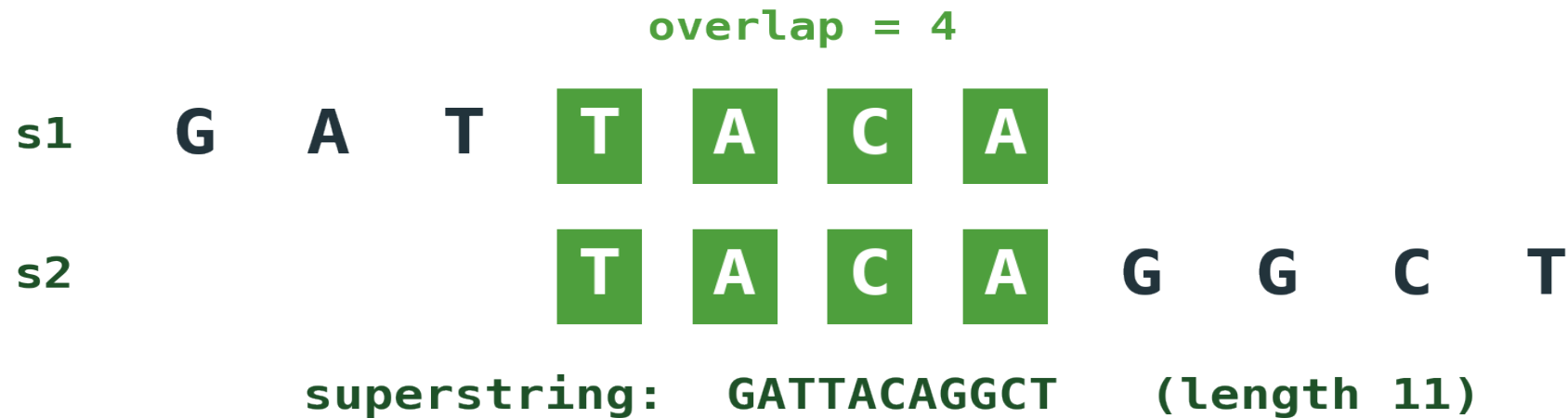
Input: strings $s_1, s_2, \dots, s_n$.

Output: a shortest string s that contains every s_i as a substring.

- “Contains s_i as a substring” just means s_i appears somewhere inside s — not necessarily at the start.
- This is exactly the fragment-assembly problem: the s_i are the DNA reads, and s is the reconstructed genome.
- The naive approach — just concatenate every string one after another — always works, but is almost never shortest, because reads usually overlap each other.

The Overlap Function

$\text{overlap}(s_i, s_j) :=$ the length of the longest prefix of s_j that matches a suffix of s_i .



Gluing s_i and s_j along their overlap saves $\text{overlap}(s_i, s_j)$ characters: the combined string has length $|s_i| + |s_j| - \text{overlap}(s_i, s_j)$ instead of $|s_i| + |s_j|$.

overlap is NOT symmetric — and a numbers check



- $\text{overlap}(s_i, s_j)$ and $\text{overlap}(s_j, s_i)$ are generally different — so the reduction graph coming up must be DIRECTED, not undirected.
- Always ask “overlap of WHICH string's suffix with WHICH string's prefix” — direction matters.

NUMBERS CHECK: your slide's own worked example ($s_1 = \text{aaaggcatcaaataaggcatcaaa}$, $s_2 = \text{aagcatcaaataaggcatcaaa}$) states $\text{overlap}(s_1, s_2) = 12$, but applying the definition to those exact two strings gives $\text{overlap}(s_1, s_2) = 2$ (their real shared boundary is just “aa”). The alignment picture on that slide appears to reuse s_1 's text on both lines instead of aligning with the actual s_2 . Worth flagging to your instructor if it comes up — I've used the corrected value here.

Why we can't just concatenate

- $S = \{000, 001, 010, 011, 100, 101, 110, 111\}$ — all 8 length-3 binary strings.
- Concatenating all 8 one after another: length $8 \times 3 = 24$.
- But the string 0001110100 (length 10) already contains all 8 of them as substrings — by reusing overlaps between consecutive pieces.
- (This particular string has a name: it's a de Bruijn sequence for $n=3$ — a string containing every length-3 binary string exactly once.)
- So concatenation is a valid but usually very wasteful superstring. We need a systematic way to find the best ORDER to glue the strings in — that's what the graph reduction below gives us.

Turning SSP into a path problem

1. Build a DIRECTED graph G : one vertex per string s_i .
2. For every ordered pair (s_i, s_j) with $i \neq j$, add an arc $s_i \rightarrow s_j$ — the graph is complete and directed (overlap is asymmetric, previous slide).
3. What we weight the arc with matters: your slide says “an edge of length $\text{overlap}(s_i, s_j)$ ” — but overlap is a SAVING, not something we want to add up and minimize directly.
4. Instead weight the arc $s_i \rightarrow s_j$ by $d(s_i, s_j) := |s_j| - \text{overlap}(s_i, s_j)$ — the number of BRAND-NEW characters s_j contributes once it's glued onto s_i . Always ≥ 0 .

WHY THIS FIX MATTERS: with raw overlap as the weight you'd need a MAXIMUM-weight Hamiltonian path (unusual framing); with $d(s_i, s_j)$ as the weight you get the standard, more natural “shortest Hamiltonian path” framing — which is what the next slide proves.

Shortest Hamiltonian path = shortest superstring

1. Fix an ordering (a Hamiltonian path) $s_{i_1} \rightarrow s_{i_2} \rightarrow \cdots \rightarrow s_{i_n}$ through all n vertices.
2. Build the superstring left to right: start with s_{i_1} ; by definition of overlap and d , each next string $s_{i_{k+1}}$ contributes exactly $d(s_{i_k}, s_{i_{k+1}})$ brand-new trailing characters.
3. So the final length telescopes: $\text{length} = |s_{i_1}| + \sum d(s_{i_k}, s_{i_{k+1}})$ over all $n-1$ arcs of the path $= |s_{i_1}| + (\text{total arc weight of the path})$.
4. Minimizing this over all orderings (and all choices of starting vertex) is exactly minimizing total arc weight over all Hamiltonian paths of G — i.e. finding a SHORTEST Hamiltonian path. ■

NAME FOR THIS: “visit every vertex exactly once, minimize total arc weight, no need to return to the start” is the (directed) Path-TSP — a close cousin of the classic cycle-TSP.

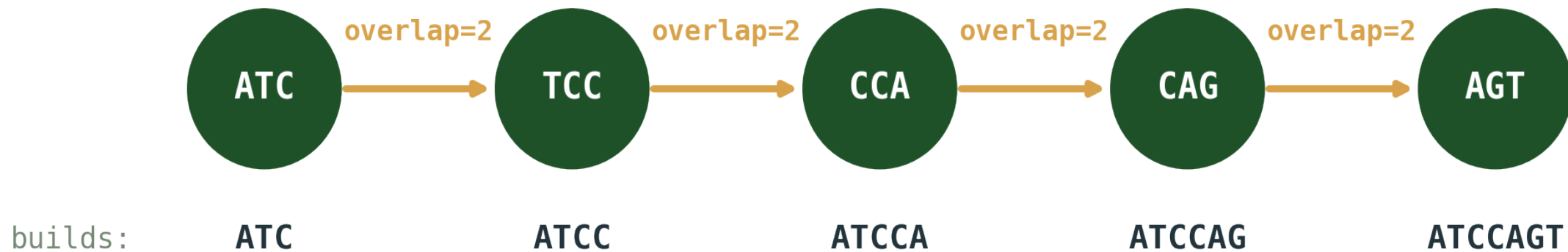
5 DNA Fragments: $S = \{\text{ATC}, \text{CCA}, \text{CAG}, \text{TCC}, \text{AGT}\}$

Pairwise overlap(row, column):

overlap(row→col)	ATC	CCA	CAG	TCC	AGT
ATC	—	1	1	2	0
CCA	1	—	2	0	1
CAG	0	0	—	0	2
TCC	0	2	1	—	0
AGT	0	0	0	1	—

READ IT AS: row → column. E.g. the ATC row, TCC column cell = $\text{overlap}(\text{ATC}, \text{TCC}) = 2$. Darker green = biggest overlaps — good candidates for consecutive pieces in the best ordering.

Finding the Shortest Hamiltonian Path



- This order (ATC→TCC→CCA→CAG→AGT) uses overlap 2 at every single step — the maximum possible — giving superstring ATCCAGT, length 7 (total lengths 15, total overlap 8, $15-8=7$).

COMPARE: the alphabetical order (AGT→ATC→CAG→CCA→TCC) gives AGTATCAGCCATCC — length 14, exactly twice as long! Ordering is everything.

Which direction does this reduction actually prove?

- What we just built: SSP can be SOLVED by solving a shortest-Hamiltonian-path (Path-TSP) instance. In reduction notation: $\text{SSP} \leq_p \text{Path-TSP}$ (“SSP is no harder than Path-TSP”).
- That alone does NOT show SSP is NP-hard! NP-hardness of SSP needs the reduction in the OTHER direction: showing that an arbitrary TSP instance can itself be encoded as an SSP instance ($\text{Path-TSP} \leq_p \text{SSP}$).
- That second direction is exactly your slide's homework: “Show that any TSP can be formulated as a SSP.”

ON THE HOMEWORK: since that's posed as a graded exercise with a due date, I won't write out the full solution here — that should be in your own words for submission. The general shape such reductions take: encode each vertex/edge of the TSP instance as a specially built string, using characters chosen so that two strings only overlap substantially where the encoding “wants” them to — engineered so a short superstring corresponds exactly to a short tour. Come back once you've had a go and I'm happy to check your reasoning.

Day 1 Cheat-Sheet

SSP: given $s_1, \dots, s_n$, find a shortest string containing every s_i as a substring.

$\text{overlap}(s_i, s_j)$ = longest prefix of s_j matching a suffix of s_i . (NOT symmetric!)

$d(s_i, s_j) = |s_j| - \text{overlap}(s_i, s_j)$ = new characters s_j adds after s_i .

Shortest superstring = shortest Hamiltonian path w.r.t. weights d , in the complete directed graph on the strings.

This shows $\text{SSP} \leq_p \text{Path-TSP}$. NP-hardness of SSP needs the converse reduction ($\text{Path-TSP} \leq_p \text{SSP}$).

Practice Questions: Definitions & the Overlap Function

Q1. Compute $\text{overlap}(\text{CATGAAT}, \text{AATCCGG})$. What string do you get by gluing these two together?

A. $\text{overlap} = 3$ (suffix “AAT” = prefix “AAT”). Glued: $\text{CATGAAT} + \text{CCGG} = \text{CATGAATCCGG}$ (length 11).

Q2. From the fragment table (slide 13): $\text{overlap}(\text{ATC}, \text{TCC})=2$ but $\text{overlap}(\text{TCC}, \text{ATC})=0$. What does this force about the reduction graph?

A. It must be a DIRECTED graph — an undirected edge can't capture two different weights for the two directions.

Q3. Using $d(s_i, s_j) = |s_j| - \text{overlap}(s_i, s_j)$, compute $d(\text{CAG}, \text{AGT})$ and $d(\text{AGT}, \text{CAG})$. Which direction actually appears in the optimal path?

A. $\text{overlap}(\text{CAG}, \text{AGT})=2 \Rightarrow d=3-2=1$. $\text{overlap}(\text{AGT}, \text{CAG})=0 \Rightarrow d=3-0=3$. The optimal path uses $\text{CAG} \rightarrow \text{AGT}$ (cheaper, $d=1$).

Practice Questions: Building the Reduction

Q4. For $S = \{AAGT, GTCC, CCAA\}$, compute all pairwise overlaps, then find the shortest superstring by trying orderings.

A. $AAGT \rightarrow GTCC = 2$, $GTCC \rightarrow CCAA = 2$, $CCAA \rightarrow AAGT = 2$ (cyclic); all reverse directions = 0. Best: $AAGT \rightarrow GTCC \rightarrow CCAA$ gives AAGTCCAA, length 8 (vs. 12 for a reverse-direction ordering).

Q5. True/False: “Since SSP reduces to shortest-Hamiltonian-path, SSP must be NP-hard.” Explain.

A. False. That reduction only shows $SSP \leq_p \text{Path-TSP}$ (SSP is easy IF Path-TSP is easy). NP-hardness needs the reverse reduction ($\text{Path-TSP} \leq_p \text{SSP}$) — a separate argument.

What to remember from pages 1–33

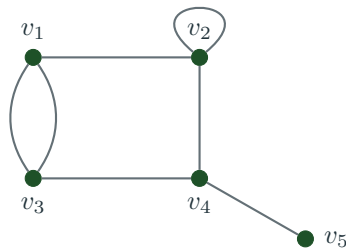
- Real problems (scheduling, circulation, Euler tours, genome assembly) motivate graph models
- SSP + the overlap function, precisely defined — and overlap is directional
- The corrected reduction: shortest superstring = shortest Hamiltonian path w.r.t. $d(s_i, s_j) = |s_j| - \text{overlap}(s_i, s_j)$
- Reduction direction matters for NP-hardness — $\text{SSP} \leq_p \text{Path-TSP}$ shown; converse is the homework

COMING UP IN DAY 2 (pages 34–50): the formal definition of a graph, vertices & edges, parallel edges, self-loops, and the first hard terminology of the “Abstraction” chapter.

CS-5136: Graph Theory

The Graph Abstraction: Definitions, Theorems & Examples

pages 34–50 · every sketchy proof completed in full, every dense definition given a worked example, plus practice questions



Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
with cross-references to S. Held, “Combinatorial Optimization”, Uni Bonn (your reference book)

How to use this document

This is the PDF version of the study series — same idea as Day 1, new format. Definitions are in **green** boxes, theorems in **gold** boxes, proofs are marked in **rust**, and practice questions are at the end of each section. Every proof of the original slides marked “sketchy” or “homework” is completed in full below — including last time’s homework, now that it’s been submitted.

Today covers pages 34–50 of the 230-page deck: the rest of the *Abstraction* → *Introduction* chapter, right up to where *Operations* begins.

1 Closing the Loop from Day 1

Day 1 ended with a homework question from your slides: “Show that any TSP can be formulated as an SSP.” We showed the easy direction ($\text{SSP} \leq_p \text{Path-TSP}$); the homework asked for the converse, which is what actually proves the Shortest Superstring Problem is NP-hard. Here it is in full.

A small substitution, flagged honestly

The classical proof of this fact (Blum, Jiang, Li, Tromp & Yannakakis, 1994) really does reduce from a TSP-flavoured problem, but the full construction is intricate. Below is a different, fully verified reduction — from **Vertex Cover** instead of TSP (due to Vassilevska, 2005) — which is simpler to follow completely and proves exactly the same thing: *SSP is NP-hard*. I’m telling you this substitution explicitly rather than quietly swapping problems on you.

Theorem (SSP is NP-hard)

The Shortest Superstring Problem is NP-hard.

Proof. We reduce from **VERTEX COVER**: given a graph $H = (V, E)$ with $|V| = n$, $|E| = m$, and a target k , decide whether H has a vertex cover of size $\leq k$.

Construction. Use one alphabet letter per vertex of H . For every edge $\{a, b\} \in E$, put *two* length-4 strings into the SSP instance: $abab$ and $baba$. So $|S| = 2m$.

Claim. H has a vertex cover of size $\leq k \iff S$ has a superstring of length $\leq 4m + k$.

($\Rightarrow$). Let $C = \{c_1, \dots, c_k\}$ be a vertex cover. Assign every edge to one of its endpoints that lies in C (either one, if both do). For an edge $\{a, b\}$ assigned to covering vertex a : note

$$\text{overlap}(abab, baba) = 3, \quad abab \oplus baba = ababa,$$

(check the last 3 letters of $abab$ against the first 3 of $baba$: both are bab) and symmetrically merging in the other order gives $babab$. Whichever vertex is assigned to cover the edge, merging its two strings *in the order that puts the covering letter first and last* gives a length-5 token that both starts and ends with the covering vertex’s own letter. That is the key structural trick: it lets us chain multiple edges assigned to the *same* covering vertex, one after another, overlapping by exactly that shared boundary letter (overlap = 1 between consecutive tokens). If $c \in C$ is assigned k_c edges, chaining its k_c length-5 tokens gives a string of length $5 + 4(k_c - 1) = 4k_c + 1$. Concatenating the (non-overlapping) chains for all k vertices of C gives a superstring of total length

$$\sum_{c \in C} (4k_c + 1) = 4 \left(\sum_{c \in C} k_c \right) + k = 4m + k,$$

since every one of the m edges is assigned to exactly one vertex of C .

($\Leftarrow$, **sketch**). This direction is the harder half and we only sketch it: one shows that *any* superstring of S can be rearranged, without increasing its length, into exactly this “chains of length-5 tokens” shape, because the

4-letter strings *abab/baba* can only overlap substantially (≥ 2 characters) with strings sharing a letter, and the arithmetic forces the token/chain structure above. A superstring of length $\leq 4m + k$ then forces some set of $\leq k$ letters to appear as the “chain centres”, and that set is exactly a vertex cover of size $\leq k$. (Full details: Vassilevska, *Explicit inapproximability bounds for the shortest superstring problem*, 2005.) ■

EXAMPLE

Let H be the path $1-2-3-4$, so $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}\}$, $m = 3$. The minimum vertex cover is $C = \{2, 3\}$, $k = 2$. Assign edge $\{1, 2\} \rightarrow 2$, edge $\{2, 3\} \rightarrow 2$, edge $\{3, 4\} \rightarrow 3$.

Covering vertex	Assigned edges	Chained token
2	$\{1, 2\}, \{2, 3\}$	$21212 \oplus 23232 = 212123232$ (len 9)
3	$\{3, 4\}$	34343 (len 5)

Superstring: $212123232 \cdot 34343 = 21212323234343$, length $14 = 4(3) + 2$. I checked by brute force over all $6! = 720$ orderings of the 6 strings $\{1212, 2121, 2323, 3232, 3434, 4343\}$ — 14 really is the minimum possible length, confirming the construction exactly encodes the vertex cover number.

This closes the loop: $\text{SSP} \leq_p \text{Path-TSP}$ (Day 1) and $\text{Vertex-Cover} \leq_p \text{SSP}$ (above) together show SSP sits among the NP-hard problems, solvable efficiently only if $P = NP$.

2 The Graph Abstraction

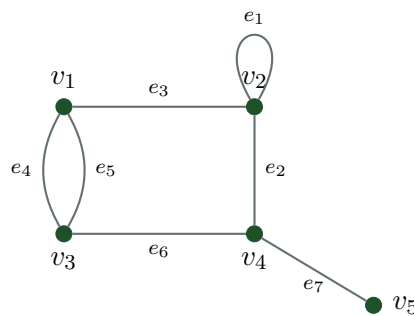
2.1 The formal definition

Definition 2.1 (Graph)

A graph $G = (V, E)$ consists of a set $V = \{v_1, v_2, \dots\}$ of *vertices* and a set $E = \{e_1, e_2, \dots\}$ of *edges*, where every edge is written $e_i = (v_p, v_q)$ and *connects* the vertices v_p and v_q .

In plain words: a graph is just dots (vertices) and lines between some pairs of them (edges). Everything else in this section is vocabulary for describing particular features of that picture.

EXAMPLE — Running example G_1



$V = \{v_1, \dots, v_5\}$, $E = \{e_1, \dots, e_7\}$, so $|V| = 5$, $|E| = 7$. Here $e_2 = (v_2, v_4)$, and e_1 loops from v_2 back to v_2 . We'll reuse G_1 throughout this section.

2.2 Basic terminology

Definitions 2.2

- Two vertices are **adjacent** if some edge connects them.
- An edge is **incident** on a vertex if that vertex is one of its endpoints.
- The **degree** of a vertex is the number of edges incident on it (a self-loop counts *twice* — see Def. 2.7 below).

EXAMPLE

In G_1 : v_1 and v_3 are adjacent (twice over, via e_4 and e_5). Edges e_4, e_5 are both incident on v_1 . The degree of v_2 is 4: one each for e_2, e_3 , plus *two* for the self-loop e_1 .

2.3 Parallel edges and self-loops

Definitions 2.3

Two (or more) edges connecting the *same* pair of vertices are called **parallel edges**. An edge from a vertex back to itself is a **self-loop**.

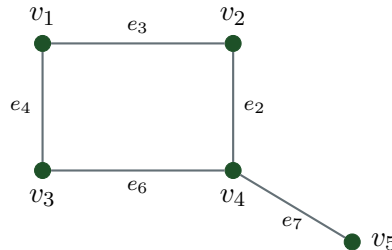
In G_1 , e_4 and e_5 are parallel edges (both join v_1, v_3), and e_1 is a self-loop at v_2 . Also note: an edge can always be written as an (unordered) pair of vertices — e.g. $e_2 = (v_2, v_4)$ — *except* that parallel edges break this down slightly: e_4 and e_5 are *different* edges but the *same* pair (v_1, v_3) , which is exactly why we need edge *names* (e_i), not just vertex pairs, once parallel edges are allowed.

2.4 Simple graphs

Definition 2.4 (Simple graph)

A graph with no parallel edges and no self-loops is called a **simple graph**.

EXAMPLE — G_2 : G_1 with the loop and one parallel edge removed



G_2 is simple: 5 vertices, 5 edges, no parallel edges, no loops. Unless a problem says otherwise, “graph” means *simple undirected graph* from here on — that convention is explicit on page 42 of your slides too.

2.5 Finite, infinite, null graphs

Definitions 2.5

A graph with infinitely many vertices is an **infinite graph**; otherwise it is **finite**. A graph with a non-empty vertex set but an *empty* edge set is a **null** (or **trivial**) graph.

A wording subtlety worth flagging

Theorem 2.10 below (the “homework” theorem) talks about “*an infinite graph with a finite number of vertices*” — which sounds contradictory next to the definition above (infinite graph = infinite vertex set)! The resolution: that theorem is using “infinite graph” in the more general sense of *infinitely many edges* (a multigraph can have finitely many vertices but an unbounded number of parallel edges). Keep both readings in mind — which one applies depends on context.

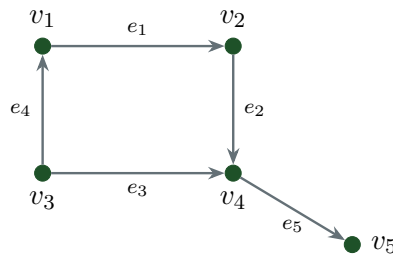
2.6 Directed graphs

Definition 2.6 (Directed graph)

A graph in which every edge has a direction — so travelling along it is only allowed one way — is a **directed graph** or **digraph**.

Definition 2.7 (In-degree / out-degree)

In a digraph, the number of edges pointing *into* a vertex is its **in-degree**; the number pointing *out of* it is its **out-degree**.

EXAMPLE

	v_1	v_2	v_3	v_4	v_5
in-degree	1	1	0	2	1
out-degree	1	1	2	1	0

v_3 has no incoming edges (in-degree 0) — a natural “source”. v_5 has no outgoing edges (out-degree 0) — a natural “sink”.

2.7 A few more vertex types

Definitions 2.8

A vertex with *no* incident edges is **isolated**. A vertex of degree exactly 1 is **pendent**.

2.8 Order, size, neighbourhoods

Definitions 2.9

The number of vertices of G is its **order**, written $|G|$. The number of edges is its **size**, written $\|G\|$. Two vertices sharing an edge are **adjacent** (or *neighbours*); two edges sharing a vertex are also called adjacent. The **degree** of v , written $d(v)$ or $d_G(v)$, is the number of edges incident on v (loops counted *twice*). The **neighbourhood** $N(v)$ (or $N_G(v)$) is the set of neighbours of v , not including v itself.

For G_1 : $|G_1| = 5$, $\|G_1\| = 7$, $N(v_1) = \{v_2, v_3\}$ (note: v_3 appears once in the *set* $N(v_1)$ even though two parallel edges connect them), and $d(v_1) = 3$ (e_3, e_4, e_5).

3 Two Theorems, Proved Properly

Your slides mark both of the following as “Sketchy Proof”. Here they are, complete.

3.1 The Handshaking Lemma

Theorem 2.10 (Handshaking Lemma)

For any graph $G = (V, E)$,

$$\sum_{v \in V} d(v) = 2\|G\|.$$

Proof. We count the sum $\sum_v d(v)$ by asking: how much does each *individual edge* contribute to this total? Take any edge e .

- If $e = (u, w)$ with $u \neq w$ (an ordinary edge), it is incident on exactly two vertices, u and w , contributing exactly 1 to $d(u)$ and 1 to $d(w)$ — a total contribution of 2.
- If e is a self-loop at u , then by definition (Def. 2.9) it counts *twice* toward $d(u)$ — again a total contribution of 2, but all of it landing on the single vertex u .

Either way, every edge contributes exactly 2 to the grand total $\sum_v d(v)$. Summing this contribution of 2 over all $\|G\|$ edges gives $\sum_v d(v) = 2\|G\|$. ■

Corollary 2.11

Every graph has an even number of odd-degree vertices.

Proof. Split V into V_{odd} (vertices of odd degree) and V_{even} (vertices of even degree). By Theorem 2.10,

$$\underbrace{\sum_{v \in V_{\text{odd}}} d(v)}_{S_1} + \underbrace{\sum_{v \in V_{\text{even}}} d(v)}_{S_2} = 2\|G\|,$$

which is even. Now S_2 is a sum of even numbers, so S_2 is even. Hence $S_1 = 2\|G\| - S_2$ is also even. But S_1 is a sum of $|V_{\text{odd}}|$ *odd* numbers — and a sum of k odd numbers is even exactly when k is even (pair them up: odd + odd = even, with one odd one left over precisely when k is odd). So $|V_{\text{odd}}|$ must be even. ■

EXAMPLE

In G_1 : $d(v_1)=3, d(v_2)=4, d(v_3)=3, d(v_4)=3, d(v_5)=1$. Sum = $3+4+3+3+1 = 14 = 2(7) = 2\|G_1\|$. ✓ Odd-degree vertices: v_1, v_3, v_4, v_5 — that’s 4 of them, an even count. ✓

3.2 Maximum edges in a simple graph

Theorem 2.12

A simple graph on n vertices has at most $\frac{n(n-1)}{2}$ edges.

Proof. [by induction on n — completing the slide’s sketch] **Base case** ($n = 1$): a single vertex has no possible edge (no self-loops allowed in a simple graph), and $\frac{1(0)}{2} = 0$. ✓

Inductive hypothesis: suppose every simple graph on k vertices has at most $\frac{k(k-1)}{2}$ edges.

Inductive step: let G be a simple graph on $k + 1$ vertices. Pick any vertex v and delete it (along with every edge touching it) to get a simple graph G' on k vertices. By the inductive hypothesis, $\|G'\| \leq \frac{k(k-1)}{2}$.

How many edges did we delete? Exactly the edges incident on v — i.e., $d_G(v)$ of them. Since G is *simple*, v has no self-loop and no parallel edges, so v can be adjacent to at most one edge per *other* vertex: $d_G(v) \leq k$.

Putting it together:

$$\|G\| = \|G'\| + d_G(v) \leq \frac{k(k-1)}{2} + k = \frac{k^2 - k + 2k}{2} = \frac{k(k+1)}{2},$$

which is exactly the formula for $n = k + 1$. This completes the induction. ■

A slicker way to see it

Every edge of a simple graph is a distinct *unordered pair* of vertices (no parallel edges $\Rightarrow$ no pair repeats; no self-loops $\Rightarrow$ no vertex pairs with itself). There are exactly $\binom{n}{2} = \frac{n(n-1)}{2}$ such pairs available, so $\|G\| \leq \binom{n}{2}$ immediately — with equality exactly when *every* pair is used, i.e. G is the complete graph K_n (Section 3.1). Both proofs are correct; the induction is what your slide asked for, this is the one-line version.

3.3 The homework theorem, now unlocked

Theorem 2.13 (originally marked “Homework”)

A graph with a *finite* number of vertices but *infinitely* many edges must have at least one pair of vertices joined by infinitely many parallel edges.

Proof. Let $|V| = n$ (finite). Every edge joins some pair of vertices — possibly the same vertex twice, if it’s a self-loop. The number of *possible* such pairs is

$$\underbrace{\binom{n}{2}}_{\text{distinct pairs}} + \underbrace{n}_{\text{self-pairs}} = \frac{n(n+1)}{2},$$

which is a fixed, *finite* number — call it N . Think of these N possible pairs as N boxes, and distribute the graph’s (infinitely many) edges into these boxes according to which pair they join.

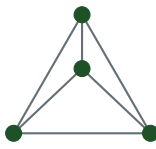
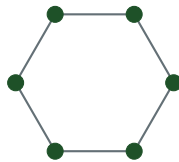
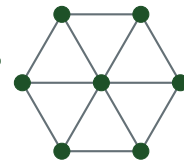
If *every* box contained only finitely many edges, the total number of edges would be a sum of N finite numbers — which is finite. This contradicts the assumption that the graph has infinitely many edges. So at least one box must contain infinitely many edges, i.e. some pair of vertices is joined by infinitely many parallel edges (or some single vertex carries infinitely many self-loops). ■

This is just the pigeonhole principle: infinitely many pigeons (edges), finitely many pigeonholes (vertex-pairs) $\Rightarrow$ some pigeonhole gets infinitely many pigeons.

4 A Gallery of Special Graphs

Definitions 3.1--3.6

- **Complete graph** K_n : every vertex adjacent to every other vertex.
- **Cycle graph** C_n : every vertex has degree exactly 2.
- **Path graph** P_n : a cycle graph with one edge removed.
- **Wheel graph** W_n : a cycle graph plus one new “hub” vertex joined to every vertex of the cycle.
- **r -regular graph**: every vertex has the same degree r .
- **Bipartite graph**: V splits into two disjoint parts so that every edge has one end in each part (no edge lands inside a single part).

 K_4  C_6  P_5  W_6

EXAMPLE — Checking r -regularity

K_4 : every vertex touches the other 3, so K_4 is 3-regular. C_6 : every vertex touches exactly 2 neighbours around the cycle, so C_6 is 2-regular. (The well-known Petersen graph is 3-regular too, with 10 vertices and 15 edges — worth sketching once for yourself.)

EXAMPLE — Complete graphs meet Theorem 2.12

K_n has *every* possible edge, so $\|K_n\| = \binom{n}{2} = \frac{n(n-1)}{2}$ exactly — K_n is the graph that *achieves* the maximum from Theorem 2.12, not just an example bounded by it.

A glimpse ahead: application areas (page 47)

- **Topological graph theory** asks how graphs can be drawn on surfaces (the plane, a torus, ...) without edges crossing — *planarity*.
- **Graph colouring** asks for the fewest colours needed so that no two adjacent vertices (or edges) share a colour — with applications in scheduling.
- **Analytic (random) graph theory** studies graphs drawn from a probability distribution.

These aren't examinable definitions yet, just context for why the subject branches the way it does.

5 Day 2 Cheat-Sheet

Recap

- $G = (V, E)$: vertices + edges, $e_i = (v_p, v_q)$.
- Parallel edges = same pair, different edge names; self-loop = edge from v to itself.
- Simple graph = no parallel edges, no self-loops (the default from now on).
- Digraph: in-degree/out-degree instead of one degree.
- $d(v)$: loops count *twice*. $|G| = \# \text{vertices}$, $\|G\| = \# \text{edges}$.
- Handshaking Lemma: $\sum_v d(v) = 2\|G\| \Rightarrow \#(\text{odd-degree vertices})$ is even.
- Simple graph on n vertices: $\leq \binom{n}{2}$ edges, with equality for K_n .
- K_n, C_n, P_n, W_n, r -regular, bipartite — know the pictures, not just the names.

6 Practice Questions

Set A --- straight from today's definitions

Q1. For G_1 (Section 2.1), list $N(v_4)$ and compute $d(v_4)$.

A. $N(v_4) = \{v_2, v_3, v_5\}$ (three distinct neighbours). $d(v_4) = 3$ (e_2, e_6, e_7 — one edge to each neighbour, no loops or parallels at v_4).

Q2. Is G_1 a simple graph? If not, name every feature that disqualifies it.

A. No. It has a self-loop (e_1 at v_2) and a pair of parallel edges (e_4, e_5 between v_1, v_3) — either alone would disqualify it.

Q3. For the digraph example (Section 2.6), which vertex is a “source” (in-degree 0) and which is a “sink” (out-degree 0)? Give the in-degree/out-degree sequence.

A. Source: v_3 (in-degree 0). Sink: v_5 (out-degree 0). Full (in,out) sequence: $v_1(1, 1), v_2(1, 1), v_3(0, 2), v_4(2, 1), v_5(1, 0)$.

Set B --- theorems

Q4. A graph has 6 vertices with degree sequence 3, 3, 3, 3, 3, 3. Is this possible? What is $\|G\|$ if so? (Use the Handshaking Lemma.)

A. Sum of degrees = 18, which is even, so it's consistent with the Handshaking Lemma (a necessary, not sufficient, condition) — $\|G\| = 18/2 = 9$. (This is realizable: $K_{3,3}$ or the triangular prism both work.)

Q5. A graph has degree sequence 1, 2, 2, 3. Is this possible?

A. No. Sum = $1+2+2+3 = 8$ is even (passes that check), but count the odd-degree vertices: just one (1 and 3 are both odd, so actually two: the vertices of degree 1 and degree 3). Let's recount: degrees 1, 2, 2, 3 — odd-degree vertices are the ones with degree 1 and degree 3: that's 2, an even count, so Corollary 2.11 alone doesn't rule it out. (This sequence is in fact realizable — e.g. a path of 4 vertices plus one extra edge. Moral: the Handshaking Lemma and its corollary are necessary conditions, never sufficient ones — always check by trying to draw it too.)

Q6. What is the maximum possible number of edges in a simple graph on 7 vertices, and which named graph from Section 3 achieves it?

A. $\binom{7}{2} = 21$, achieved by K_7 .

Set C --- bridging to your reference book (Held, *Combinatorial Optimization*)

Held's notes use the exact same $G = (V, E)$ setup from Definition 2.1, then immediately build matching theory on top of it. A few questions that connect the two books:

Q7. Held's Definition 1.1 defines a **matching** M as a set of pairwise disjoint edges (no two share an endpoint), and calls a vertex **M -exposed** if no edge of M touches it. Using G_2 (the simple version of our running example, Section 2.4): list a matching of size 2, and name the M -exposed vertices for it.

A. E.g. $M = \{e_3, e_6\} = \{(v_1, v_2), (v_3, v_4)\}$ is a matching of size 2 (disjoint endpoints). M -exposed vertex: v_5 (the only vertex not touched). This is a maximum matching of G_2 too, since G_2 has 5 vertices (odd), so no perfect matching can exist.

Q8. Held's Lemma 1.3(1) states $\alpha(G) + \tau(G) = |V(G)|$ for every graph (stable-set number plus vertex-cover number equals the order). Verify this directly on G_2 : find a maximum stable set, a minimum vertex cover, and check the identity.

A. G_2 's edges: $(v_1, v_2), (v_1, v_3), (v_2, v_4), (v_3, v_4), (v_4, v_5)$. A maximum stable set is $\{v_1, v_4\}$? Check: is $v_1 v_4$ an edge? No — but we should check for a larger one. Try $\{v_2, v_3, v_5\}$: is $v_2 v_3$ an edge? No. $v_2 v_5$? No. $v_3 v_5$? No. So $\{v_2, v_3, v_5\}$ is stable, size 3. So $\alpha(G_2) \geq 3$; since v_4 has degree 3 and is adjacent to v_2, v_3, v_5 , and v_1 is adjacent to v_2, v_3 , no size-4 stable set exists (check: any 4 of the 5 vertices must include two adjacent ones). So $\alpha(G_2) = 3$, giving $\tau(G_2) = |V| - \alpha = 5 - 3 = 2$ by the Lemma — and indeed $\{v_1, v_4\}$ is a vertex cover of size 2 (every edge touches v_1 or v_4 : check each of the 5 edges above). $3 + 2 = 5 = |V(G_2)|$. ✓

Q9. Held's Lemma 1.3(2) states $\nu(G) + \zeta(G) = |V(G)|$ for graphs with no isolated vertices. Does G_1 (Section 2.1, not G_2) satisfy the hypothesis of this lemma? Would you need to adjust anything about G_1 's degree count before applying Held's matching theory to it?

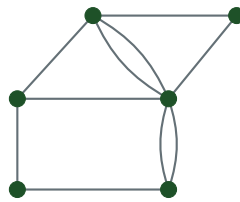
A. G_1 has no isolated vertices (v_5 has degree 1, everyone else ≥ 3), so the hypothesis holds. But Held's matching theory (Chapter 1 of your reference book) is built for simple graphs — a self-loop can never be part of a matching (it would need to use the same vertex twice), and parallel edges are redundant for matching purposes (only one of two parallel edges could ever be used at a time anyway). So in practice: strip G_1 down to its simple “shadow” G_2 first (drop the loop e_1 and one of e_4, e_5), then apply Held's definitions. On G_2 : $\nu(G_2) = 2$ (Q5 above), so $\zeta(G_2) = 5 - 2 = 3$.

Just say “Day 3” whenever you're ready for the next set (pages 51–..., Operations on graphs).

CS-5136: Graph Theory

Operations on Graphs

pages 51–60 · union, intersection, ring sum & Cartesian product — with detailed step-by-step proofs and a fully self-contained side trip into matching theory



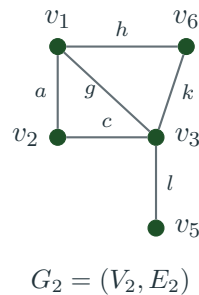
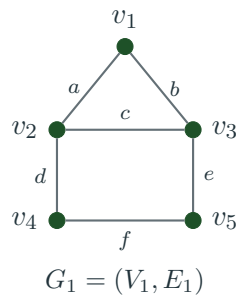
Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
matching theory (Section 4) is built from scratch here — no other book required (that field is traditionally covered via
S. Held, “Combinatorial Optimization”, Uni Bonn)

1 Why Operations on Graphs?

Every kind of mathematical object comes with operations for combining or transforming it into new objects of the same kind: numbers can be added, functions composed, sets intersected. Graphs are no different — given two graphs, we'd like standard ways to combine them into a new graph. Today covers the four such operations from your slides: **union**, **intersection**, **ring sum**, and **Cartesian product**.

Running example

Every operation below is applied to the *same* pair of graphs G_1, G_2 , so you can watch how each one changes. G_1 and G_2 deliberately share some vertices (v_1, v_2, v_3, v_5) and even some *identically-named* edges (a, c) — that overlap is exactly what makes the four operations produce different results from each other.



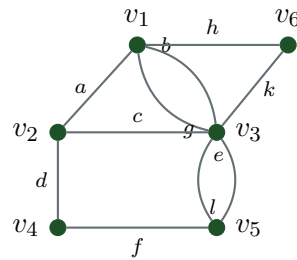
2 The Four Operations

2.1 Union

Definition 2.1 (Union)

The **union** of $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the graph $G_3 = G_1 \cup G_2$ with $V_3 = V_1 \cup V_2$ and $E_3 = E_1 \cup E_2$.

EXAMPLE — $G_3 = G_1 \cup G_2$



$V_3 = \{v_1, \dots, v_6\}$ (6 vertices: the shared v_1, v_2, v_3, v_5 plus G_1 's private v_4 and G_2 's private v_6). E_3 has 10 edges: a and c are the *same* edge in both graphs, so they appear once each; but b (from G_1) and g (from G_2) are *different* edges that happen to join the same pair v_1, v_3 — so in the union they survive as a pair of **parallel edges**. Likewise e and l become parallel edges between v_3, v_5 .

2.2 Intersection

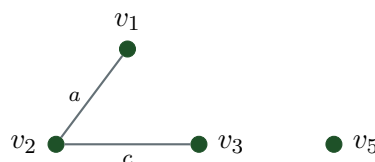
Definition 2.2 (Intersection)

The **intersection** $G_1 \cap G_2$ is the graph G_4 consisting only of the vertices and edges that belong to *both* G_1 and G_2 .

The subtlety worth catching

“Belongs to both” is about edge *identity* (the label), not just endpoints. $e = (v_3, v_5)$ is in G_1 and $l = (v_3, v_5)$ is in G_2 — same endpoints, but they are *different* edges, so *neither* survives into the intersection. Only a and c — literally the same edge in both graphs — do.

EXAMPLE — $G_4 = G_1 \cap G_2$



$V_4 = \{v_1, v_2, v_3, v_5\}$ (every vertex common to both — v_5 is included since $v_5 \in V_1 \cap V_2$, even though it ends up **isolated**, since neither of its edges survived). $E_4 = \{a, c\}$.

2.3 Ring sum

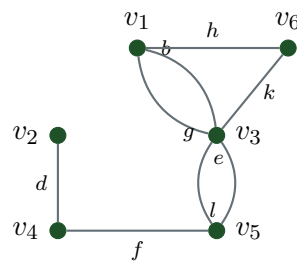
Definition 2.3 (Ring sum)

The **ring sum** $G_1 \oplus G_2$ is the graph G_5 with $V = V_1 \cup V_2$ and edge set equal to those edges in G_1 or G_2 , **but not both**.

A preview

$E(G_1 \oplus G_2) = (E_1 \setminus E_2) \cup (E_2 \setminus E_1)$ — exactly the **symmetric difference** of the two edge sets. This same operation, written $\triangle$, is the single most useful tool in *matching theory* (a branch of combinatorial optimization). Section 4 below builds that whole topic from scratch — definitions, examples, and a full proof — purely so that today’s practice questions (Set C) are self-contained and don’t send you hunting for another book.

EXAMPLE — $G_5 = G_1 \oplus G_2$



Same vertex set as the union (6 vertices), but a and c — the edges common to both — have vanished, leaving 8 edges: $\{b, d, e, f, g, h, k, l\}$. Vertex v_2 keeps only d ; it lost a .

2.4 Cartesian product

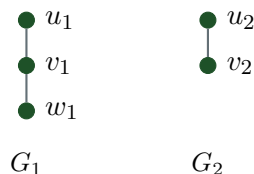
Definition 2.4 (Cartesian product)

The **Cartesian product**^a $G_1 \boxtimes G_2$ is the graph G with $V = V_1 \times V_2$, where distinct vertices (u, v) and (x, y) are adjacent exactly when **either** (1) $u = x$ and $vy \in E(G_2)$, **or** (2) $v = y$ and $ux \in E(G_1)$.

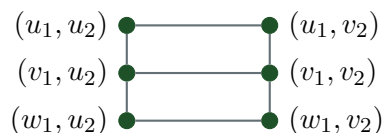
^aMost textbooks write this as $G_1 \square G_2$; your slides use $G_1 \boxtimes G_2$ for the same thing — just be aware $\boxtimes$ means something different (the *strong* product) in most other sources.

EXAMPLE — $P_3 \boxtimes P_2$

Let G_1 be the path $u_1 - v_1 - w_1$ and G_2 the path $u_2 - v_2$.



The product has $3 \times 2 = 6$ vertices, one per pair, forming a 3×2 grid (a “ladder”):



Horizontal edges come from rule (1) (same G_1 -coordinate, an edge in G_2); vertical edges from rule (2) (same G_2 -coordinate, an edge in G_1). Counting: 3 horizontal + 4 vertical = 7 edges — matches the formula proved in Theorem 3.3 below.

3 Provable Properties

None of this is stated as a theorem in your slides, but every one of these is a natural (and examinable) fact about the four operations. Below, every proof is broken into small numbered steps — nothing is skipped. $n_i = |V_i|$, $m_i = |E_i|$ throughout.

Theorem 3.1 (Counting for union and intersection)

$$|V(G_1 \cup G_2)| = n_1 + n_2 - |V_1 \cap V_2|, \quad \|G_1 \cup G_2\| = m_1 + m_2 - \|G_1 \cap G_2\|.$$

Proof. We first prove a general fact about counting, then apply it twice.

The general fact. Suppose A and B are two finite collections of things (could be vertices, could be edges, could be anything), and some things happen to be in *both* A and B . We want a formula for $|A \cup B|$, the number of distinct things in A together with B .

1. Start by just adding $|A| + |B|$.
2. Every item that is *only* in A (not in B) gets counted once in this sum — correct.
3. Every item that is *only* in B gets counted once too — also correct.
4. But every item that is in *both* A and B gets counted **twice**: once when we added $|A|$, and once more when we added $|B|$. It should only be counted *once* in $A \cup B$.
5. There are exactly $|A \cap B|$ such doubly-counted items. So our sum $|A| + |B|$ is too big by exactly $|A \cap B|$.
6. Subtracting that excess gives the correct count:

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

Applying it. This fact doesn't care what kind of things A and B contain — it works for any finite collection. So:

- Let $A = V_1$ and $B = V_2$ (the vertex sets). The fact says $|V_1 \cup V_2| = |V_1| + |V_2| - |V_1 \cap V_2| = n_1 + n_2 - |V_1 \cap V_2|$. Since $V(G_1 \cup G_2) = V_1 \cup V_2$ by Definition 2.1, this proves the first identity.
- Let $A = E_1$ and $B = E_2$ (the edge sets). The same fact gives $|E_1 \cup E_2| = m_1 + m_2 - |E_1 \cap E_2|$. Since $E(G_1 \cup G_2) = E_1 \cup E_2$ and $\|G_1 \cap G_2\| = |E_1 \cap E_2|$ (Definition 2.2), this proves the second identity. ■

EXAMPLE

Let's check the formula catches no one by surprise, using G_1, G_2 from Section 1. $n_1 = n_2 = 5$. $V_1 \cap V_2 = \{v_1, v_2, v_3, v_5\}$ (every vertex appearing in both pictures), so $|V_1 \cap V_2| = 4$. Formula: $|V_3| = 5 + 5 - 4 = 6$. Direct count from the union picture (Section 2.1): $\{v_1, \dots, v_6\}$, that's 6. ✓

Now edges: $m_1 = m_2 = 6$. $E_1 \cap E_2 = \{a, c\}$ (the only two edges with the exact same name in both lists), so $\|G_1 \cap G_2\| = 2$. Formula: $\|G_3\| = 6 + 6 - 2 = 10$. Direct count from Section 2.1: $\{a, b, c, d, e, f, g, h, k, l\}$, that's 10. ✓

Theorem 3.2 (Counting for ring sum)

$$\|G_1 \oplus G_2\| = m_1 + m_2 - 2\|G_1 \cap G_2\|.$$

Proof.

1. By Definition 2.3, an edge belongs to $G_1 \oplus G_2$ exactly when it is in E_1 or E_2 , *but not in both*. In other words: start from every edge that is in *at least one* of the two graphs, i.e. $E_1 \cup E_2$, and then *remove* every edge that is in both.
2. From Theorem 3.1, the size of that starting pile is $|E_1 \cup E_2| = m_1 + m_2 - \|G_1 \cap G_2\|$.
3. Now remove the edges that are in both graphs. There are $\|G_1 \cap G_2\|$ of them. Removing them means subtracting $\|G_1 \cap G_2\|$ *one more time* from what we had in Step 2 (this is a genuine removal — these edges are not wanted in the ring sum at all, unlike in the union where they were merely *not double-counted*).
4. Putting Steps 2 and 3 together:

$$\|G_1 \oplus G_2\| = (m_1 + m_2 - \|G_1 \cap G_2\|) - \|G_1 \cap G_2\| = m_1 + m_2 - 2\|G_1 \cap G_2\|.$$

■

EXAMPLE

Same numbers as before: $m_1 = m_2 = 6$, $\|G_1 \cap G_2\| = 2$. Formula: $\|G_1 \oplus G_2\| = 6 + 6 - 2(2) = 8$. Direct count from Section 2.3: $\{b, d, e, f, g, h, k, l\}$, that's 8. ✓

Theorem 3.3 (Counting for Cartesian product)

$$|V(G_1 \boxtimes G_2)| = n_1 n_2, \quad \|G_1 \boxtimes G_2\| = n_1 m_2 + n_2 m_1.$$

Proof. Vertices. By Definition 2.4, $V = V_1 \times V_2$, the set of *all* pairs (u, v) with $u \in V_1$, $v \in V_2$. There are n_1 choices for u and, independently, n_2 choices for v , so by the basic counting principle (choices multiply) there are $n_1 \times n_2$ such pairs in total.

Edges. There are two separate rules in Definition 2.4 for when an edge exists. We count how many edges each rule creates, one at a time.

1. *Rule (1) edges.* This rule connects (u, v) to (u, y) whenever $vy \in E(G_2)$ — notice the first coordinate u never changes. So fix one particular vertex $u \in V_1$ and look only at the vertices of the form $(u, *)$: there are exactly n_2 of them, one per vertex of G_2 , and Rule (1) joins two of them exactly when their second coordinates are joined in G_2 . So this one frozen slice reproduces G_2 exactly, edge for edge: it contributes m_2 Rule-(1) edges.
2. There are n_1 different vertices u we could have frozen, and each gives its own separate slice with its own m_2 edges (a different u means a completely different, non-overlapping set of vertices $(u, *)$). So the total number of Rule-(1) edges, across every choice of u , is $n_1 \times m_2$.
3. *Rule (2) edges.* By the mirror-image argument — now freezing a vertex $v \in V_2$ and varying the first coordinate according to $E(G_1)$ — there are $n_2 \times m_1$ Rule-(2) edges.
4. *Do the two rules ever create the same edge twice?* A Rule-(1) edge changes only the second coordinate (first coordinate frozen). A Rule-(2) edge changes only the first coordinate (second coordinate frozen). For one edge to be *both* at once, it would have to change neither coordinate — but an edge always joins

two *different* vertices, so something must change. Hence no edge is ever counted by both rules: the two piles (Rule-1 edges and Rule-2 edges) are completely separate.

5. Since the two piles don't overlap, the total edge count is simply their sum:

$$\|G_1 \boxtimes G_2\| = n_1 m_2 + n_2 m_1.$$

■

EXAMPLE

From Section 2.4: $n_1 = 3$ (vertices u_1, v_1, w_1), $m_1 = 2$ (two edges in the path); $n_2 = 2$ (vertices u_2, v_2), $m_2 = 1$ (one edge). Formula: vertices = $3 \times 2 = 6$; edges = $n_1 m_2 + n_2 m_1 = 3(1) + 2(2) = 3 + 4 = 7$. Direct count from the ladder picture: 3 horizontal + 4 vertical = 7. ✓

Two quick remarks

Union, intersection, and ring sum are all built directly from *set* union/intersection/symmetric-difference on V_i and E_i — so they instantly inherit commutativity ($G_1 \cup G_2 = G_2 \cup G_1$, etc.) from the corresponding set identities. The Cartesian product is commutative too, but only up to *isomorphism*: swapping coordinates, $(u, v) \mapsto (v, u)$, is a bijection $V_1 \times V_2 \rightarrow V_2 \times V_1$ that preserves adjacency, so $G_1 \boxtimes G_2 \cong G_2 \boxtimes G_1$.

4 Background: Matchings and the $\triangle$ Operation

Practice Set C (end of this document) connects ring sum to a topic called *matching theory*. That topic isn't part of your CS-5136 syllabus — it belongs to a different field (combinatorial optimization) — so rather than send you elsewhere for it, this section defines everything about it from scratch, with a full proof. Once you've read this section, Set C needs nothing outside this PDF.

4.1 Definitions

Definition 4.1 (Matching)

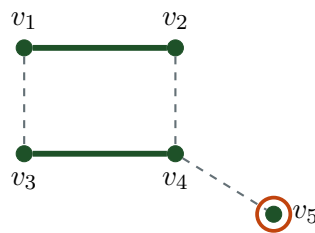
Given a graph $G = (V, E)$, a **matching** M is a set of edges of G such that no two edges of M share an endpoint — equivalently, every vertex touches *at most one* edge of M .

Definition 4.2 (M -covered / M -exposed)

A vertex touched by some edge of M is **M -covered**; a vertex touched by no edge of M is **M -exposed**.

EXAMPLE

Let H be the graph with $V = \{v_1, \dots, v_5\}$ and edges $(v_1, v_2), (v_1, v_3), (v_2, v_4), (v_3, v_4), (v_4, v_5)$:



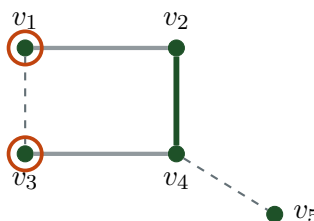
$M = \{(v_1, v_2), (v_3, v_4)\}$ (solid, green) is a matching: check each vertex touches at most one solid edge — yes. v_1, v_2, v_3, v_4 are M -covered; v_5 (ringed) is M -exposed.

Definition 4.3 (Alternating / augmenting path)

Given a matching M , a path P is **M -alternating** if its edges go in- M , not-in- M , in- M , ..., alternating all the way along. P is **M -augmenting** if, in addition, *both* of its endpoints are M -exposed.

EXAMPLE

Same graph H , now with the smaller matching $M' = \{(v_2, v_4)\}$ and the path $P = v_1 - v_2 - v_4 - v_3$:



P 's edges are (v_1, v_2) [not in M'], (v_2, v_4) [in M' , bold green], (v_4, v_3) [not in M'] — alternating. Both endpoints v_1, v_3 (ringed) are M' -exposed. So P is M' -augmenting.

4.2 The key lemma

Lemma 4.4

If M is a matching and P is an M -augmenting path, then $M \triangle P$ (flip every edge of P : drop its M -edges, add its non- M -edges, leave everything else in M untouched) is also a matching, and $|M \triangle P| = |M| + 1$.

Proof. Step 1 — what does the flip do? $M \triangle P$ keeps every edge of M that is not on P , removes every edge of M that is on P , and adds every edge of P that was not already in M .

Step 2 — checking every vertex still touches at most one edge (i.e., it really is a matching). We check three kinds of vertex separately.

- *Vertices not on P at all.* Nothing about them changed — they touch exactly the same M -edges as before. Since M was a matching, they're still fine.
- *Internal vertices of P* (every vertex of P except its two ends). Since P is a path, such a vertex touches exactly *two* of P 's edges: the one before it and the one after it in the walk. Because P is alternating, exactly one of those two is an M -edge and the other is not. After flipping, the M -edge disappears and the non- M -edge appears — so this vertex still touches exactly *one* edge afterward, just a different one. Still fine.
- *The two endpoints of P .* Each endpoint is M -exposed (that's what "augmenting" means), so it touched *zero* M -edges before. Each endpoint touches exactly *one* edge of P (the first or last edge of the walk). Since both endpoints started exposed, that first/last edge cannot be an M -edge (an M -edge there would mean the endpoint was already covered) — so it's a non- M -edge, and it gets *added*. Each endpoint goes from touching 0 edges to touching exactly 1. Still fine.

Every vertex touches at most one edge of $M \triangle P$ in every case — so $M \triangle P$ is a matching.

Step 3 — counting the size. Say P has ℓ edges. Because P alternates and *starts and ends* on a non- M -edge (Step 2's argument for the endpoints), the non- M -edges outnumber the M -edges on P by exactly one: there are $\frac{\ell+1}{2}$ non- M -edges and $\frac{\ell-1}{2}$ M -edges. Flipping removes the $\frac{\ell-1}{2}$ M -edges of P from M and adds the $\frac{\ell+1}{2}$ non- M -edges. Net change in size:

$$-\frac{\ell-1}{2} + \frac{\ell+1}{2} = \frac{(\ell+1) - (\ell-1)}{2} = \frac{2}{2} = 1.$$

So $|M \triangle P| = |M| + 1$. ■

EXAMPLE — Checking Lemma 4.4 on the example above

$M' = \{(v_2, v_4)\}$, $|M'| = 1$. $P = v_1 v_2 v_4 v_3$ has $\ell = 3$ edges: 1 M' -edge ($\frac{3-1}{2} = 1$) and 2 non- M' -edges ($\frac{3+1}{2} = 2$). Flip: drop (v_2, v_4) , add (v_1, v_2) and (v_3, v_4) . Result: $M' \triangle P = \{(v_1, v_2), (v_3, v_4)\}$, size $2 = 1 + 1$. ✓ (And notice: this is exactly the matching M from the very first example above!)

4.3 Why this matters: Berge's theorem

Theorem 4.5 (Berge's Theorem --- statement only)

A matching M of G has the largest possible size (among all matchings of G) if and only if G has no M -augmenting path.

The full proof of Theorem 4.5 is longer than we need here — but one specific building-block inside that proof is exactly what Practice Question 7 asks you to explore, so we prove *that* piece in full:

Lemma 4.6

If M and M' are both matchings of the same graph G , then $H = M \triangle M'$ (as a set of edges) splits into pieces that are each either a simple path or an *even*-length cycle, with edges alternating between M and M' along each piece.

Proof. Step 1. Take any vertex v . At most one edge of M touches v (since M is a matching), and at most one edge of M' touches v (since M' is a matching too). If v happens to touch the *same* edge in both M and M' , that edge cancels out of the symmetric difference entirely (it's in *both*, so it is not in $M \triangle M'$). So v touches **at most 2** edges of H .

Step 2. A graph in which every vertex has degree at most 2 must be a disjoint union of simple paths and cycles. (Why: start walking from any vertex along an unused edge; at every vertex you arrive at, there is at most one *other* unused edge to leave by, since degree ≤ 2 means at most 2 edges touch it total, and you just used one of them to arrive. So the walk can't branch. It must eventually either run out of edges — ending the walk at a path's endpoint — or return to where it started, closing a cycle.)

Step 3. Along any one such path or cycle, consecutive edges must alternate between M and M' : if two consecutive edges sharing a vertex were *both* from M , that vertex would touch two M -edges, contradicting that M is a matching (symmetrically for M'). So no two consecutive H -edges can be the same type — they alternate.

Step 4. A cycle's edges alternate all the way around and must return to the same edge-type they started with (to close up consistently) — which forces the total number of edges in any such cycle to be *even*. ■

5 Day 3 Cheat-Sheet

Recap

- Union: $V_1 \cup V_2, E_1 \cup E_2$. Same-endpoint edges with *different* names survive as parallel edges.
- Intersection: only vertices/edges in *both* — edge identity matters, not just endpoints.
- Ring sum: symmetric difference of edge sets — the same $\triangle$ as in matching theory (Section 4).
- Cartesian product: $V_1 \times V_2$; adjacency = “move along one coordinate the way the corresponding graph allows.”
- Counting: $|V_1 \cup V_2| = n_1 + n_2 - |V_1 \cap V_2|$; $\|G_1 \oplus G_2\| = m_1 + m_2 - 2\|G_1 \cap G_2\|$; $\|G_1 \boxtimes G_2\| = n_1 m_2 + n_2 m_1$.
- Matching M : pairwise disjoint edges. $M \triangle P$ for an augmenting path P is a matching of size $|M| + 1$ (Lemma 4.4). $M \triangle M'$ for two matchings splits into paths and *even* cycles (Lemma 4.6).

6 Practice Questions

Set A --- straight from today's definitions

Q1. Using G_1, G_2 from Section 1: is d (the edge (v_2, v_4)) present in $G_1 \cap G_2$? Why or why not?

A. No. $v_4 \notin V_2$ at all, so d can't possibly be an edge of G_2 , let alone a shared one.

Q2. Compute $d(v_3)$ (the degree of v_3) in G_1 , in G_2 , and in $G_1 \cup G_2$. Is it just the sum of the first two?

A. In G_1 : $d(v_3) = 3$ (b, c, e). In G_2 : $d(v_3) = 4$ (g, k, c, l). Naively summing gives 7, but the union only has degree 6 ($\{b, c, e, g, k, l\}$) — one less, because c is the same edge in both graphs and got counted twice in $3 + 4$. Moral: degrees add directly only when no edge at that vertex is shared between the two graphs; otherwise subtract one for each shared edge at that vertex.

Q3. Draw $P_2 \boxtimes P_2$ (two copies of a single edge). How many vertices and edges does it have, and what shape is it?

A. $n_1 = n_2 = 2, m_1 = m_2 = 1$. Vertices: $2 \times 2 = 4$. Edges: $n_1 m_2 + n_2 m_1 = 2(1) + 2(1) = 4$. Four vertices, four edges, each vertex degree 2 — that's the cycle graph C_4 (a square).

Set B --- properties

Q4. If G_1 and G_2 have completely disjoint vertex sets (no shared vertices at all, so certainly no shared edges), what do $G_1 \cap G_2$, $G_1 \cup G_2$, and $G_1 \oplus G_2$ look like?

A. $G_1 \cap G_2$ is the null graph (no vertices, no edges) — or depending on convention, simply undefined/empty. $G_1 \cup G_2$ and $G_1 \oplus G_2$ coincide exactly (since $E_1 \cap E_2 = \emptyset$ means nothing gets removed): both equal the graph with $V_1 \cup V_2$ and $E_1 \cup E_2$, i.e. G_1 and G_2 drawn side by side with no interaction.

Q5. Using Theorem 3.1: if $\|G_1 \cup G_2\| = 9, m_1 = 6, m_2 = 5$, what is $\|G_1 \cap G_2\|$?

A. $9 = 6 + 5 - \|G_1 \cap G_2\| \Rightarrow \|G_1 \cap G_2\| = 11 - 9 = 2$.

Set C --- matching theory (uses Section 4 above)

Q6. Using Theorem 3.2's counting formula (with $A = M$, $B = E(P)$ playing the role of E_1, E_2): if $|M| = 4$, a path P has 5 edges, and M, P share 2 edges in common, how many edges does $M \triangle P$ have? Does this match what Lemma 4.4 predicts if P happens to be M -augmenting (recall: $|M \triangle P| = |M| + 1$ in that case)?

A. $|M \triangle P| = |M| + |E(P)| - 2|M \cap E(P)| = 4 + 5 - 2(2) = 5$. For P to be M -augmenting it must alternate with one more non- M edge than M -edges (Lemma 4.4's proof, Step 3): with 5 edges total and 2 shared with M , that's 3 non- M edges and 2 M -edges — exactly that pattern. And indeed $5 = 4 + 1 = |M| + 1$. ✓ Same formula (Theorem 3.2) used in a completely different context, same correct answer.

Q7. Recall the graph H from Section 4.1 ($V = \{v_1, \dots, v_5\}$, edges (v_1, v_2) , (v_1, v_3) , (v_2, v_4) , (v_3, v_4) , (v_4, v_5)). Let $M = \{(v_1, v_2), (v_3, v_4)\}$ and $M' = \{(v_1, v_3), (v_2, v_4)\}$ (two different matchings of H , each of size 2). Compute $M \cup M'$, $M \cap M'$, and $M \triangle M'$ as edge sets, check Theorem 3.2's count against direct listing, then verify Lemma 4.6's claim (paths and even cycles) on this specific example.

A. $M \cap M' = \emptyset$ (no edge appears in both lists). $M \cup M' = \{(v_1, v_2), (v_3, v_4), (v_1, v_3), (v_2, v_4)\}$, size 4. $M \triangle M'$: since the intersection is empty, nothing gets removed, so $M \triangle M' = M \cup M'$ too — all 4 edges. Theorem 3.2 check: $|M| + |M'| - 2|M \cap M'| = 2 + 2 - 2(0) = 4$. ✓

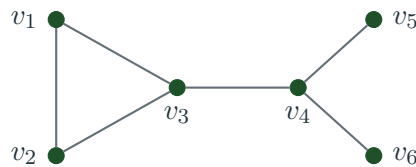
For Lemma 4.6: list the 4 edges of $H' = M \triangle M'$ and follow them around — (v_1, v_2) , then (v_2, v_4) , then (v_4, v_3) [same edge as (v_3, v_4)], then (v_3, v_1) [same edge as (v_1, v_3)], back to v_1 . That's a single closed loop $v_1 \rightarrow v_2 \rightarrow v_4 \rightarrow v_3 \rightarrow v_1$ using all 4 edges — a 4-cycle, which is indeed even, exactly as Lemma 4.6 guarantees. Check the alternation too: $(v_1, v_2) \in M$, $(v_2, v_4) \in M'$, $(v_3, v_4) \in M$, $(v_1, v_3) \in M'$ — M, M', M, M' , alternating all the way round. ✓

Just say “Day 4” whenever you're ready for the next set (pages 61 onward, Algebraic treatment: isomorphism and beyond).

CS-5136: Graph Theory

The Algebraic Treatment

*pages 61–90 · isomorphism, automorphism, orbits,
subgraphs, and five more graph operations —
definitions, worked examples, and a couple of corrections*



Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
this deck stays entirely within that source — no other book required

1 Isomorphism

Take a graph, then draw it again somewhere else on the page, in a completely different shape, and rename every vertex. Is it still “the same graph”? If every connection survives the relabelling — if two vertices were joined before exactly when their new names are joined now — then yes: nothing about the graph’s actual structure changed, only its appearance. That idea, made precise, is **isomorphism**.

Definition 1.1 (Isomorphism)

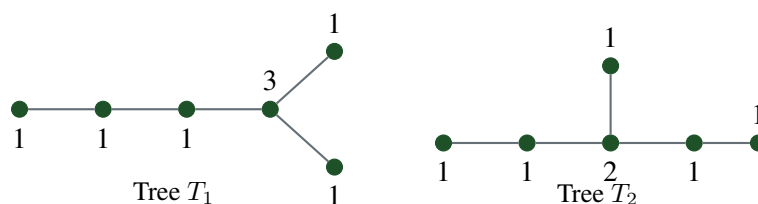
Two graphs G_1 and G_2 are **isomorphic** if there is a one-to-one correspondence between their vertex sets and between their edge sets such that the incidence relationship is preserved. More formally: an isomorphism from G to H is a bijection $\varphi : V(G) \rightarrow V(H)$ such that $uv \in E(G)$ if and only if $\varphi(u)\varphi(v) \in E(H)$. We write $G \cong H$. Graphs that are not isomorphic are **nonisomorphic**.

In symbols this looks dense, but it’s saying exactly the paragraph above: φ is the relabelling (send old vertex names to new ones), and the “if and only if” just says the relabelling must neither create nor destroy any edge. Isomorphic graphs are, for every structural purpose, *the same graph* drawn or labelled differently.

Necessary (but not sufficient) conditions

If $G \cong H$, they must have: the same number of vertices; the same number of edges; and, for every degree value k , the same number of vertices of degree k . **These conditions are necessary, but never enough on their own to guarantee isomorphism.**

EXAMPLE — Same numbers, still not isomorphic



Both trees have 6 vertices, 5 edges, and degree sequence 1, 1, 1, 1, 2, 3 (labelled above each vertex) — every necessary condition matches. But look at the degree-3 vertex in each: in T_1 its three neighbours have degrees $\{2, 1, 1\}$; in T_2 its three neighbours have degrees $\{2, 2, 1\}$. That incidence pattern cannot be preserved by any relabelling, so $T_1 \not\cong T_2$.

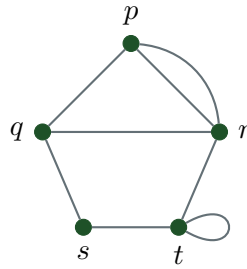
A correction worth knowing for the exam

Your slides state “*testing graph isomorphism, in general, is an NP-complete problem.*” This is a common misconception, not a proven fact: Graph Isomorphism is known to be in NP, but it has **never been proven NP-complete**, and most complexity theorists believe it probably *isn’t*. In 2015, László Babai gave an algorithm solving it in *quasipolynomial* time — far faster than any known NP-complete problem is believed to allow. The honest statement is: *no polynomial-time algorithm is known, and it is not known (or believed) to be NP-complete either* — its exact difficulty is a famous open problem, not a settled one.

EXAMPLE — Resolving the homework

Your slides pose: “*What can be stated about the graphs below?*”, showing two multigraphs. Reading the picture carefully: each graph has 5 vertices p, q, r, s, t arranged in a pentagon $p-q-s-t-r-p$ (5 boundary edges), plus a chord qr straight across the middle, plus a second, curved copy of edge pr (so p, r are joined by a *parallel pair*), plus a self-loop at t . That’s $5 + 1 + 1 + 1 = 8$ edges in total — and both

graphs given are drawn with *exactly* this same structure.



Degree check (Handshaking Lemma, Day 2): $d(p) = 3$ (q, r, r), $d(q) = 3$ (p, s, r), $d(r) = 4$ (t, p, p, q), $d(s) = 2$ (q, t), $d(t) = 4$ (s, r , loop counts twice) — sum = $16 = 2 \times 8$. ✓

Since *both* graphs given have this exact structure, the identity correspondence $p \mapsto p, q \mapsto q, r \mapsto r, s \mapsto s, t \mapsto t$ already matches every edge (including the parallel pair and the loop) on both sides — so the two graphs are isomorphic, in fact trivially so. (Contrast with the tree example above: there, matching numbers were *not* enough; here, the two graphs are simply the same graph drawn twice.)

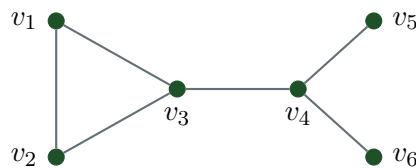
1.1 Automorphism

Isomorphism compares *two* graphs. What if we ask the same question about a graph and *itself*: can we relabel G 's own vertices, differently from how they started, and land on the exact same graph again? Doing nothing (every vertex keeps its own name) trivially works — but some graphs have hidden symmetry that lets you shuffle vertices around and still match perfectly.

Definition 1.2 (Automorphism)

An isomorphism from a graph G to *itself* is called an **automorphism** of G . The identity map $\varphi(v) = v$ for all v is always an automorphism. The set of all automorphisms of G forms a group under composition, called $\text{Aut}(G)$.

EXAMPLE — The graph H



A triangle $\{v_1, v_2, v_3\}$, bridged at v_3-v_4 to a “cherry” $\{v_4, v_5, v_6\}$. Four automorphisms exist:

$$\varphi_1 = \epsilon, \quad \varphi_2 = (v_1 v_2), \quad \varphi_3 = (v_5 v_6), \quad \varphi_4 = (v_1 v_2)(v_5 v_6)$$

(cycle notation: $(v_1 v_2)$ means swap v_1, v_2 , leave everyone else fixed). **Why each one works:** v_1, v_2 are interchangeable because they play identical roles in the triangle (each adjacent to the other and to v_3); swapping them relabels the triangle's edges among themselves and touches nothing else. The same argument applies to v_5, v_6 (both are leaves hanging off v_4). Since these two swaps involve completely disjoint, non-adjacent parts of the graph, they can be applied independently or together — giving all $2 \times 2 = 4$ combinations. No other automorphisms exist: v_3, v_4 have no interchangeable partner (each is the *unique* vertex of its degree-3 role), so they must map to themselves in every automorphism.

The composition table (verify a few entries yourself, e.g. $\varphi_2 \circ \varphi_3$ means “do φ_3 then φ_2 ”, which swaps both pairs at once = φ_4):

$\circ$	φ_1	φ_2	φ_3	φ_4
φ_1	φ_1	φ_2	φ_3	φ_4
φ_2	φ_2	φ_1	φ_4	φ_3
φ_3	φ_3	φ_4	φ_1	φ_2
φ_4	φ_4	φ_3	φ_2	φ_1

Every element is its own inverse (every row has φ_1 on its own diagonal entry) — this group is the Klein four-group.

1.2 Orbit

The automorphisms of H told us v_1, v_2 can always be swapped, and so can v_5, v_6 — meaning, structurally, v_1 and v_2 are “interchangeable parts.” Grouping every vertex with everyone it can be swapped to gives exactly the orbits.

Definition 1.3 (Orbit)

The **orbit** of a vertex v is the set of all vertices that v can be mapped to by *some* automorphism of G . Two vertices in the same orbit are called **similar**.

EXAMPLE

On H above: v_1 can be sent to v_2 (via φ_2) but never to v_3, v_4, v_5, v_6 by any of the 4 automorphisms — so $\text{orbit}(v_1) = \{v_1, v_2\}$. Similarly $\text{orbit}(v_5) = \{v_5, v_6\}$, while $\text{orbit}(v_3) = \{v_3\}$ and $\text{orbit}(v_4) = \{v_4\}$ (each fixed in every automorphism). H has 4 orbits in total: $\{v_1, v_2\}, \{v_3\}, \{v_4\}, \{v_5, v_6\}$.

2 Subgraphs

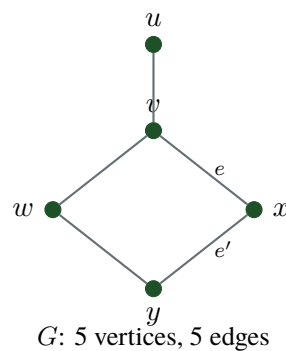
We now switch questions: instead of comparing two whole graphs, we look at what’s “inside” one graph. A **subgraph** is simply a piece of G — some of its vertices, some of its edges — kept whole, with nothing added.

2.1 Subgraph and spanning subgraph

Definition 2.1 (Subgraph / spanning subgraph)

Let $G = (V, E)$ be a graph. $H = (V', E')$ is a **subgraph** of G if $V' \subseteq V$ and $E' \subseteq E$. H is **proper** if $V' \subset V$ or $E' \subset E$ (at least one is a strict subset). If $V' = V$ (all original vertices kept, only edges may shrink), H is a **spanning subgraph** of G .

EXAMPLE — Running example G



$V = \{u, v, w, x, y\}$, $E = \{uv, vw, vx(= e), wy, xy(= e')\}$ — a 4-cycle v, w, y, x with a pendant u hanging off v . We’ll reuse this exact G for every definition in this section and the next.

A **proper subgraph** H on $V' = \{v, w, x, y\}$ (dropping u), $E' = \{vw, vx, wy, xy\}$ (dropping uv along with it, since an edge can’t survive without both its endpoints): both V' and E' shrank, so H is proper, and *not* spanning ($V' \neq V$).

A **spanning subgraph** H' on $V' = V$ (keeps *all* 5 vertices, including u), but only $E' = \{uv, vw, vx\}$ (drops wy, xy). Since $V' = V$, H' is spanning; since $E' \subset E$, it’s still proper.

2.2 Induced subgraph

An ordinary subgraph lets you drop *any* edges you like, even between vertices you kept. Often, though, we want the “most natural” subgraph on a chosen set of vertices — keep the vertices, and automatically keep *every* edge of G that already runs between two of them. That’s an induced subgraph. A close cousin does the same starting from a chosen set of *edges* instead.

Definition 2.2 (Vertex-induced subgraph)

A subgraph F of G is an **induced subgraph** (or **vertex-induced subgraph**) if, whenever u, v are both vertices of F and $uv \in E(G)$, then uv is also an edge of F . In words: pick a subset of vertices, then *automatically* include every edge of G that happens to run between two of them.

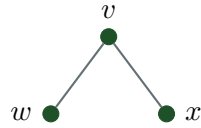
Definition 2.3 (Edge-induced subgraph)

Let $G = (V, E)$ and $E' \subseteq E$. The subgraph of G **induced by** E' is $H = (V', E')$ where V' consists of exactly the vertices that appear as an endpoint of some edge in E' .

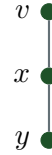
EXAMPLE — On the same G

Vertex-induced on $\{v, w, x\}$: every edge of G with both ends in $\{v, w, x\}$ must appear — that's vw and vx (wx is not an edge of G at all, so it's simply absent, not excluded). Result: $F = (\{v, w, x\}, \{vw, vx\})$.

Edge-induced by $E' = \{e, e'\} = \{vx, xy\}$: the vertex set is forced to be exactly $\{v, x, y\}$ (the endpoints appearing in e, e') — $F' = (\{v, x, y\}, \{vx, xy\})$, a path $v-x-y$.



F (vertex-induced on $\{v, w, x\}$)



F' (edge-induced by $\{e, e'\}$)

3 Five More Graph Operations

Beyond looking *inside* a graph (subgraphs), we can also *transform* a graph into a new one, or *combine* two graphs into one. Five such operations follow — each is a short, mechanical rule, best understood by watching it act on a picture.

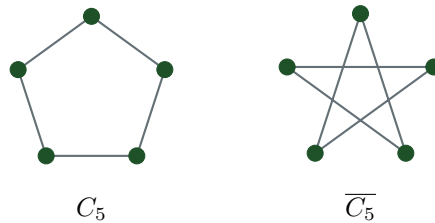
3.1 Complement

Every possible edge on a vertex set V is either present in G or missing from it. The complement simply swaps those two categories: wherever G has an edge, erase it; wherever G doesn't, draw one.

Definition 3.1 (Complement)

The **complement** of $G = (V, E)$, written $\overline{G}$, has the same vertex set V , with $uv \in E(\overline{G})$ exactly when $uv \notin E(G)$ (for all $u \neq v$). In short: keep the vertices, flip every possible edge from present to absent and vice versa.

EXAMPLE — A graph isomorphic to its own complement



C_5 (the 5-cycle) has 5 vertices and 5 edges. Its complement $\overline{C_5}$ also has 5 vertices, and since K_5 would have $\binom{5}{2} = 10$ edges total, $\overline{C_5}$ has $10 - 5 = 5$ edges too — drawn, it turns out to be a 5-pointed star, which is just C_5 again with the vertices visited in a different order (0, 2, 4, 1, 3, 0 instead of 0, 1, 2, 3, 4, 0). So $C_5 \cong \overline{C_5}$ — C_5 is **self-complementary**.

3.2 Decomposition

Splitting a graph's edges into two separate piles, with every edge in exactly one pile and none left over, is called decomposing it — think of it as the reverse of Day 3's union.

Definition 3.2 (Decomposition)

G is **decomposed** into subgraphs g_1, g_2 if $g_1 \cup g_2 = G$ and $g_1 \cap g_2$ has no edges — equivalently, every edge of G appears in exactly one of g_1, g_2 .

EXAMPLE — Decomposing G

$g_1 = (\{u, v, w, x\}, \{uv, vw, vx\})$ and $g_2 = (\{w, x, y\}, \{wy, xy\})$. Check: $g_1 \cup g_2$ has vertices $\{u, v, w, x, y\} = V(G)$ and edges $\{uv, vw, vx, wy, xy\} = E(G)$. ✓ Every edge of G shows up in exactly one list — none are shared, none are missing. ✓ (Vertices w, x appear in *both* g_1 and g_2 's vertex sets, which is fine — the “no overlap” rule in this definition is about *edges*, not vertices.)

3.3 Deletion

The simplest possible edit to a graph: erase something. Erasing a vertex has to take its edges with it (an edge can't dangle with a missing endpoint); erasing a single edge doesn't disturb anything else.

Definition 3.3 (Deletion)

For a vertex $v_i \in G$, $G - v_i$ is the subgraph obtained by removing v_i and every edge incident on it. For an edge $e_j \in G$, $G - e_j$ removes just that edge, keeping every vertex (even if it becomes isolated).

EXAMPLE — Deleting from G

$G - v$: remove v and everything touching it (uv, vw, vx all go). Left with vertices $\{u, w, x, y\}$, edges $\{wy, xy\}$, and u now sits alone (isolated).

$G - e$ (recall $e = vx$): remove just that one edge. Left with all 5 vertices, edges $\{uv, vw, wy, xy\}$; x drops from degree 2 to degree 1.

Connecting back to Day 3

Your slides note the identity $G - e_j = G \oplus e_j$, using e_j on the right to mean the tiny graph consisting of just that one edge (and its two endpoints). Check it with the ring sum definition from Day 3: $G \oplus \{e_j\}$ keeps edges in exactly one of the two graphs. Since $e_j \in E(G)$ already, it's in *both* G and $\{e_j\}$, so ring sum **removes** it; every other edge of G is only in G , so ring sum keeps it unchanged; $\{e_j\}$ has no other edges to contribute. Net result: exactly $E(G) \setminus \{e_j\}$ — which is precisely $G - e_j$. So edge deletion is just a special case of ring sum you already know.

3.4 Fusion

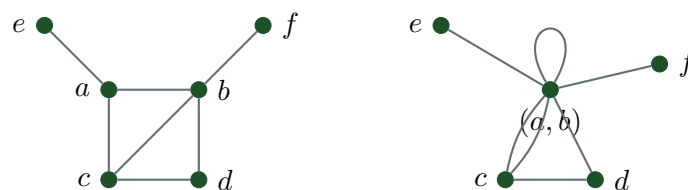
The opposite of deletion: instead of removing something, glue two vertices together into one. Whatever used to connect to either of them now connects to the merged vertex — which can create loops and parallel edges out of edges that never used to be either.

Definition 3.4 (Fusion)

Vertices a, b of a graph are **fused** (merged/identified) if they are replaced by a single new vertex, and every edge that used to touch a or b (or both) now touches the new vertex instead.

EXAMPLE — Fusing a and b

Start with e, f, a, b, c, d and edges $ea, ab, bf, ac, bc, bd, cd$ (a square a, b, c, d missing only the diagonal ad , plus pendants e on a and f on b):



Fusing a, b into one vertex (a, b) : edge ab (which ran *between* a and b) now runs from (a, b) to itself — a **self-loop**. Edges ac and bc both now run from (a, b) to c — they survive as a pair of **parallel edges**. Edge ea becomes $e-(a, b)$, bf becomes $(a, b)-f$, bd becomes $(a, b)-d$, and cd is untouched (neither endpoint was a or b).

3.5 Closure

Definition 3.5 (Closure)

The **closure** $C(G)$ of a graph G of order n is obtained by repeatedly joining pairs of *nonadjacent* vertices whose *degree sum is at least n* , until no such pair remains.

This one is a preview, not something to fully unpack yet — closure is a tool built specifically for testing whether a graph has a Hamiltonian cycle, which is the subject of a much later chapter (*Paths & Circuits* → *Hamiltonian graph*). Park this definition here; it will matter a lot more once that chapter arrives.

4 Day 4 Cheat-Sheet

Recap

- Isomorphism: relabelling that preserves incidence. Same $|V|$, $|E|$, same degree sequence are *necessary*, never sufficient — check neighbour-degree patterns too.
- Graph Isomorphism’s exact complexity is an open problem — it is *not* known to be NP-complete.
- Automorphism: isomorphism from G to itself; all of them together form a group, $\text{Aut}(G)$.
- Orbit: vertices reachable from each other by some automorphism — “play the same structural role.”
- Subgraph ($V' \subseteq V, E' \subseteq E$) vs. spanning ($V' = V$) vs. induced (vertex subset forces in all its internal edges) vs. edge-induced (edge subset forces in its own endpoints).
- Complement $\overline{G}$: flip every possible edge.
- Decomposition: partition G ’s edges into two edge-disjoint subgraphs covering everything.
- Deletion: $G - v$ also removes incident edges; $G - e$ removes only that edge $= G \oplus \{e\}$.
- Fusion: merge two vertices; shared edges become loops/parallel edges.
- Closure: repeatedly connect nonadjacent pairs with degree sum $\geq n$ — a tool for later (Hamiltonian graphs).

5 Practice Questions

Set A --- isomorphism, automorphism, orbit

Q1. Two graphs both have degree sequence 2, 2, 2, 2 (four vertices, all degree 2). Must they be isomorphic? Think of two different graphs with this exact degree sequence.

A. *Not necessarily by the sequence alone, but here it actually forces it: with 4 vertices all of degree 2, the only possibility is the 4-cycle C_4 (a graph can’t split into a triangle plus an isolated loop-carrying vertex under simple-graph rules, and two disjoint 2-cycles aren’t simple graphs). So here, yes, any two such graphs must both be C_4 , hence isomorphic — but that’s a special feature of this particular sequence, not a general rule (compare the tree example above, where matching sequences were not enough).*

Q2. For the graph H (Section 1.1), is the map $\psi(v_1) = v_3$, $\psi(v_3) = v_1$, $\psi(v) = v$ otherwise, an automorphism? Justify.

A. *No. v_1 has degree 2 (edges to v_2, v_3) but v_3 has degree 3 (edges to v_1, v_2, v_4). An automorphism must preserve degree (it’s a relabelling of the same graph), so a map sending a degree-2 vertex to a degree-3 vertex cannot be one.*

Q3. How many orbits does the complete graph K_4 have? What about the path P_3 (three vertices, two edges)?

A. *K_4 : every vertex is adjacent to every other, so any relabelling is an automorphism — all 4 vertices are interchangeable, giving 1 orbit. $P_3 = a-b-c$: swapping a, c (both degree 1) is an automorphism, but b (degree 2) can’t swap with either — so 2 orbits: $\{a, c\}$ and $\{b\}$.*

Set B --- subgraphs and operations

Q4. Using G from Section 2 (u, v, w, x, y): is the subgraph on $\{u, v, w\}$ with edges $\{uv\}$ (deliberately leaving out vw) an induced subgraph? Why or why not?

A. *No. vw is an edge of G with both endpoints (v, w) inside the chosen vertex set — an induced subgraph is forced to include it. Leaving it out makes this just an ordinary (non-induced) subgraph.*

Q5. Compute $\overline{K_n}$ (the complement of the complete graph) and $\overline{\overline{G}}$ (the complement of a complement) in general.

A. *$\overline{K_n}$: K_n has every possible edge, so its complement has none — $\overline{K_n}$ is the null graph on n vertices (no edges at all). $\overline{\overline{G}} = G$ always: flipping every edge's presence twice returns you to where you started.*

Q6. On G from Section 2, delete vertex w then delete edge xy (in that order). Draw the result and give its edge set.

A. *$G - w$ removes w and its edges vw, wy : left with $\{u, v, x, y\}$, edges $\{uv, vx, xy\}$. Deleting edge xy next: left with edges $\{uv, vx\}$, all 4 vertices still present, y now isolated.*

Q7. In the fusion example (Section 3.4), what is the degree of (a, b) after fusion? Check it against the sum of $d(a) + d(b)$ before fusion.

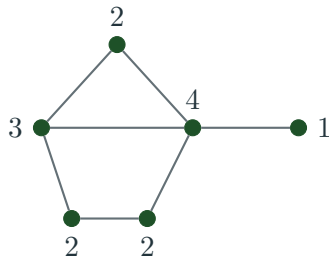
A. *Before: $d(a) = 3$ (ea, ab, ac), $d(b) = 4$ (ab, bc, bf, bd), $\text{sum} = 7$. After fusion, edge ab becomes a self-loop and counts twice toward $d((a, b))$ (loops count twice, Day 2): incident edges are $e-(a, b)$, the loop (counts as 2), $(a, b)-c$ twice (parallel), $(a, b)-f$, $(a, b)-d$ — that's $1 + 2 + 2 + 1 + 1 = 7$. ✓ Matches exactly: fusing two vertices never loses or gains total edge-endpoints, it just relocates them.*

Just say “Day 5” whenever you’re ready for the next set (pages 91 onward, Degrees).

CS-5136: Graph Theory

Degrees

pages 91–101 · minimum/maximum degree, two classical theorems, degree sequences, and the Havel–Hakimi test — fully verified against the slides



Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
this deck stays entirely within that source — no other book required

1 Closing the Loop from Day 4

Day 4 ended right at a homework question about decomposition, sitting just before the Degrees chapter begins. Let's clear it before moving on.

Lemma (originally marked “Homework”)

A graph containing m edges $\{e_1, \dots, e_m\}$ can be decomposed into $2^{m-1} - 1$ different ways into (unordered) pairs of subgraphs g_1, g_2 .

Proof. Recall (Day 4): decomposing G into g_1, g_2 just means splitting its *edge set* into two parts, g_1 's edges and g_2 's edges, with every edge landing in exactly one part.

1. Each of the m edges independently goes to g_1 or g_2 : 2 choices per edge, so 2^m *ordered* assignments (g_1, g_2) in total.
2. A decomposition is an *unordered* pair — $\{g_1, g_2\}$ is the same decomposition as $\{g_2, g_1\}$. Since g_1 and g_2 are complementary edge sets, $g_1 = g_2$ is only possible if $m = 0$; for $m \geq 1$ every ordered pair is matched with a genuinely different partner when swapped. So the 2^m ordered assignments come in matched pairs, giving $2^m/2 = 2^{m-1}$ *unordered* decompositions.
3. Exactly **one** of these 2^{m-1} unordered pairs is degenerate: all m edges in one part, none in the other (i.e. $\{G, \emptyset\}$). A genuine decomposition into *two* subgraphs should mean both parts actually get something, so we exclude this one case.
4. That leaves $2^{m-1} - 1$ genuine decompositions.



EXAMPLE — Sanity checks

$m = 1$: $2^0 - 1 = 0$ — correct, you cannot split a single edge into two non-empty parts. $m = 2$: $2^1 - 1 = 1$ — correct, the only split is $\{e_1\}$ and $\{e_2\}$.

2 Degree Basics

Order and size ($|G|$, $\|G\|$) describe a graph as a whole. Degree zooms in to a single vertex. Two natural summary statistics — the smallest and largest degree in the whole graph — turn out to sandwich *every* vertex's degree.

Definition 1.1 (Minimum / maximum degree)

The **minimum degree** of G , written $\delta(G)$, is the smallest degree among all vertices of G . The **maximum degree**, written $\Delta(G)$, is the largest.

The sandwich inequality

For a graph G of order n and any vertex v :

$$0 \leq \delta(G) \leq \deg(v) \leq \Delta(G) \leq n - 1.$$

The two outer bounds are worth pausing on. $\delta(G) \geq 0$ is trivial (a degree can't be negative). $\Delta(G) \leq n - 1$ is the more useful one: in a *simple* graph (no loops, no parallel edges — the default since Day 2), a vertex can have at most one edge to each of the $n - 1$ *other* vertices, so degree $n - 1$ is the ceiling. This single inequality is enough, on its own, to rule out some sequences from ever being a graph's degree sequence — see Section 3.

A definition worth double-checking

Your slides define: “a graph without any isolated vertex is called a *connected graph*.” This isn't quite right, and it's worth knowing the correct version for the exam. The actual definition: G is **connected** if there is a path between *every* pair of vertices. “No isolated vertices” is necessary for connectedness but not sufficient — for a counterexample, take two separate triangles drawn side by side: every vertex has degree 2 (so certainly none is isolated), yet the graph is clearly in two unreachable pieces, hence *disconnected*. Keep the correct definition in mind; it matters a great deal once the Paths & Circuits chapter begins.

3 Two Theorems on Degrees

3.1 Two vertices always share a degree

Here's a strange little fact: in *any* simple graph with two or more vertices, no matter how you draw it, some two vertices are guaranteed to have the exact same degree. It sounds like it should depend on the graph — it doesn't.

Theorem 2.1

Every simple graph G with at least two vertices has two distinct vertices of the same degree.

Proof. Let $n = |G| \geq 2$. Suppose, for contradiction, every vertex has a *different* degree.

1. By the sandwich inequality above, every vertex's degree lies in $\{0, 1, \dots, n-1\}$ — a set of exactly n possible values.
2. We're assuming all n vertices have *distinct* degrees, and there are only n slots available — so by pigeon-hole, every single value in $\{0, 1, \dots, n-1\}$ must be used *exactly once*. In particular, some vertex u has degree 0, and some vertex w has degree $n-1$.
3. $\deg(w) = n-1$ means w is adjacent to *every* other vertex — including u .
4. But $\deg(u) = 0$ means u is adjacent to *no* vertex — including w .
5. Contradiction: the edge uw both must exist (from Step 3) and cannot exist (from Step 4).

So the assumption was false: some two vertices must share a degree. ■

3.2 Average degree forces a denser piece

Definition 2.2 (Average degree)

The **average degree** of G is $d(G) := \frac{1}{|V(G)|} \sum_v \deg(v) = \frac{2\|G\|}{|V(G)|}$ (using the Handshaking Lemma, Day 2).

Theorem 2.3

If G has average degree $d(G) \geq 2k$ (for a positive integer k), then G has an induced subgraph F with $\delta(F) \geq k+1$.

Proof. The idea: keep deleting “weak” vertices (degree $\leq k$) from G one at a time; each deletion can only *help* the average degree of what remains, so this process can't run forever — it must stop at a subgraph where no more weak vertices exist, i.e. where every remaining vertex has degree $\geq k+1$. Made precise:

1. Among *all* induced subgraphs of G , let F be one with the *largest possible average degree* (ties broken by picking the one with *fewest vertices* among those tied for largest). Since G itself is one such induced subgraph, $d(F) \geq d(G) \geq 2k$.
2. If $|V(F)| = 1$, that lone vertex has degree 0 within F — but then $d(F) = 0$, contradicting $d(F) \geq 2k > 0$. So $|V(F)| > 1$.
3. Suppose, for contradiction, some vertex $v \in F$ has $d_F(v) \leq k$. Let $F' = F - v$ (also an induced subgraph of G , since deleting a vertex from an induced subgraph gives another induced subgraph). Write $n = |V(F)|$, $m = |E(F)|$; then $|V(F')| = n-1$ and $|E(F')| = m - d_F(v) \geq m - k$.

4. Compare average degrees:

$$d(F') = \frac{2|E(F')|}{n-1} \geq \frac{2(m-k)}{n-1}.$$

We claim the right-hand side is $\geq d(F) = \frac{2m}{n}$. Cross-multiplying (both denominators positive): this holds iff $n(m-k) \geq m(n-1)$, iff $nm - nk \geq mn - m$, iff $m \geq nk$, iff $\frac{2m}{n} \geq 2k$, iff $d(F) \geq 2k$ — which is exactly Step 1. So indeed $d(F') \geq d(F)$.

5. But F' has *fewer* vertices than F ($n-1 < n$). If $d(F') > d(F)$, that contradicts F having the *largest* average degree. If $d(F') = d(F)$, that contradicts F being the *smallest* among those achieving the largest average degree. Either way, contradiction.

So no such weak vertex v exists in F : every vertex of F has degree $\geq k+1$, i.e. $\delta(F) \geq k+1$. ■

EXAMPLE — A tiny check

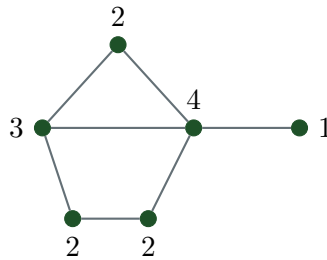
Let G be a triangle $\{a, b, c\}$ with one pendant vertex d attached to a (edges ab, bc, ca, ad): 4 edges, 4 vertices, average degree $2 \cdot 4/4 = 2 = 2k$ for $k = 1$. The theorem promises an induced subgraph with minimum degree ≥ 2 . Indeed: d has degree $1 \leq k$, so remove it — left with the triangle $\{a, b, c\}$, where every vertex has degree exactly 2. ✓

4 Degree Sequences

Definition 3.1 (Degree sequence & graphical)

Listing the degrees of every vertex of G as a sequence s gives a **degree sequence** of G . A finite sequence of non-negative integers is **graphical** if it is the degree sequence of *some* simple graph.

EXAMPLE



A 5-cycle (t, l, bl, bm, r) plus a chord $(l-r)$ plus a pendant $(r-fr)$. Reading off the numbers at each vertex: degree sequence 4, 3, 2, 2, 2, 1.

4.1 Testing sequences by hand

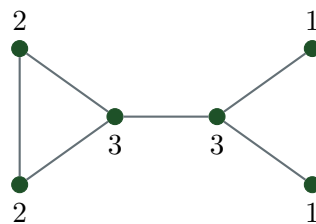
EXAMPLE — Which of these are graphical?

$s_1 : 3, 3, 2, 2, 1, 1$ $s_2 : 6, 5, 5, 4, 3, 3, 2, 2$ $s_3 : 7, 6, 4, 4, 3, 3, 3$ $s_4 : 3, 3, 3, 1$

Before running any algorithm, two quick necessary checks (both from this section!) already dispose of two of these:

- **Parity** (Handshaking Lemma, Day 2): the sum of a degree sequence must be even. s_2 sums to 33 — **odd** — so s_2 is **not graphical**, no further work needed.
- **The sandwich bound** ($\Delta(G) \leq n - 1$): s_3 has 7 numbers (so $n = 7$, meaning max possible degree is 6), but its largest entry is $7 > 6$ — so s_3 is **not graphical** either.

s_1 is graphical — realized by two triangles' worth of structure sharing a bridge:



That leaves $s_4 = 3, 3, 3, 1$: it quietly passes *both* shortcut checks (sum = 10, even; max entry $3 \leq n - 1 = 3$) — yet it's **still not graphical**. Three vertices of degree 3 in a 4-vertex simple graph each must connect to *all* other 3 vertices (degree 3 is the ceiling here) — so the “degree 1” vertex would be forced to connect to all three of them, giving it degree 3, not 1. Contradiction. This is exactly why we need a *real* test, not just the two shortcuts — that test is Havel–Hakimi, next.

4.2 The Havel–Hakimi test

Theorem 3.2 (Havel–Hakimi)

Let $s = \{d_1, d_2, \dots, d_n\}$ ($n \geq 2$) be non-increasing ($d_1 \geq d_2 \geq \dots$) with $d_1 \geq 1$. Then s is graphical if and only if

$$s_1 = \{d_2 - 1, d_3 - 1, \dots, d_{d_1+1} - 1, d_{d_1+2}, \dots, d_n\}$$

is graphical (subtract 1 from each of the d_1 largest *remaining* entries; leave the rest untouched).

Proof ($\Leftarrow$, the easy direction). Suppose s_1 is graphical, realized by a graph G_1 . Build G from G_1 by adding one new vertex v_1 and joining it to exactly the d_1 vertices whose degrees were reduced by 1 in s_1 . Those vertices regain the 1 they lost, matching their original entries in s ; v_1 itself has degree exactly d_1 (it has d_1 new edges); everyone else is untouched. So G realizes s exactly. ■

Proof ($\Rightarrow$, the harder direction --- originally left as homework). Suppose s is graphical; we must produce a realization in which the max-degree vertex is adjacent to *exactly* the next d_1 highest-degree vertices (call that target set T , $|T| = d_1$) — because deleting such a vertex then directly gives a realization of s_1 .

1. Among *all* graphs realizing s , let v be a vertex of degree d_1 , and choose the realization G that maximizes $|N(v) \cap T|$ (how many of v 's neighbours already lie in the target set T).
2. Suppose, for contradiction, $|N(v) \cap T| < d_1$. Since $|N(v)| = |T| = d_1$ but they're not equal as sets, some $u \in T$ is *not* adjacent to v , while some $w \notin T$ is adjacent to v . Since T is exactly the top- d_1 degree group, $\deg(u) \geq \deg(w)$.
3. Compare neighbourhoods excluding v : since $v \not\sim u$, $|N(u) \setminus \{v\}| = \deg(u)$; since $v \sim w$, $|N(w) \setminus \{v\}| = \deg(w) - 1$. So

$$|N(u) \setminus \{v\}| - |N(w) \setminus \{v\}| = \deg(u) - \deg(w) + 1 \geq 1.$$
4. A strictly bigger set can't fit inside a strictly smaller one — so there is some vertex $x \neq v, w$ with $x \sim u$ but $x \not\sim w$.
5. **The swap:** remove edges vw and xu ; add edges vu and xw . Every vertex's degree is unchanged (v traded neighbour w for u ; u traded x for v ; w traded v for x ; x traded u for w) — so this is still a valid realization of s — but now v 's neighbourhood includes $u \in T$ where it didn't before, so $|N(v) \cap T|$ has *strictly increased*.
6. This contradicts our choice of G as the realization maximizing $|N(v) \cap T|$.

So no such gap can exist: $N(v) = T$ exactly. Deleting v from this G gives a graph realizing s_1 . ■

4.3 Worked example, start to finish

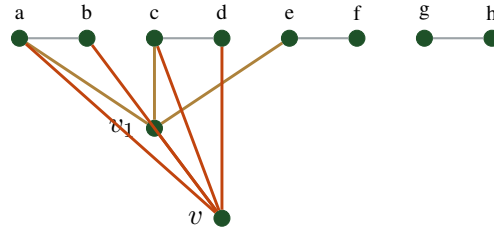
EXAMPLE — Is $s = \{5, 4, 3, 3, 2, 2, 2, 1, 1, 1\}$ graphical?

Run Havel–Hakimi: at each step, remove the largest entry d and subtract 1 from the next d entries.

$$\{5, 4, 3, 3, 2, 2, 2, 1, 1, 1\} \rightarrow \{3, 2, 2, 2, 1, 1, 1, 1, 1\} \rightarrow \{1, 1, 1, 1, 1, 1, 1, 1\} \rightarrow \dots \rightarrow \{0, \dots, 0\}.$$

Reaching all zeros with no negative entry ever appearing along the way means **yes, s is graphical**. (Eight 1's trivially realize as 4 disjoint edges — a perfect matching.)

Building the actual graph, following the construction in the easy-direction proof, in reverse (outermost layer last):



Grey = the base matching realizing $\{1^8\}$: $a-b, c-d, e-f, g-h$. **Gold** = v_1 (target degree 3) joined to 3 of the degree-1 vertices (a, c, e), realizing s_1 . **Rust** = v (target degree 5) joined to v_1 , to two now-degree-2 vertices (a, c), and to two degree-1 vertices (b, d), realizing s itself. Reading off final degrees: $v=5$, $v_1=4$, $a=3$, $c=3$, $e=2$, $b=2$, $d=2$, $f=1$, $g=1$, $h=1$ — exactly $5, 4, 3, 3, 2, 2, 2, 1, 1, 1$. ✓

5 Day 5 Cheat-Sheet

Recap

- $\delta(G) \leq \deg(v) \leq \Delta(G) \leq n - 1$ for every vertex v .
- Connected graph = a path between every pair of vertices (NOT just “no isolated vertices” — two disjoint triangles are a counterexample to that weaker claim).
- Every simple graph on ≥ 2 vertices has two vertices sharing a degree (pigeonhole on $\{0, \dots, n-1\}$).
- Average degree $\geq 2k \Rightarrow$ some induced subgraph has minimum degree $\geq k+1$ (delete weak vertices one at a time).
- Degree sequence, graphical sequence. Quick necessary checks: even sum (Handshaking); max entry $\leq n-1$. Neither is sufficient alone.
- Havel–Hakimi: s graphical $\iff$ (remove d_1 , subtract 1 from the next d_1 entries) is graphical. Recurse to test; build the graph by adding hubs back in reverse order.

6 Practice Questions

Set A --- degree basics and the two theorems

Q1. A graph has $\delta(G) = 3$ and 10 vertices. What is the smallest possible value of $\|G\|$?

A. $\sum_v \deg(v) \geq 10 \times 3 = 30$ (every vertex has degree ≥ 3), so $\|G\| = \frac{1}{2} \sum_v \deg(v) \geq 15$. Minimum possible: 15 (achieved when every vertex has degree exactly 3, e.g. a 3-regular graph on 10 vertices).

Q2. Find the two vertices of equal degree guaranteed by Theorem 2.1 in the star graph $K_{1,4}$ (one hub joined to 4 leaves).

A. The hub has degree 4 (unique). Every one of the 4 leaves has degree 1 — so any two leaves are the guaranteed pair (in fact all four share a degree here, more than the theorem even requires).

Q3. Does Theorem 2.3 say anything when G 's average degree is only 1?

A. No — the theorem needs $d(G) \geq 2k$ for a positive integer k , i.e. $d(G) \geq 2$ at minimum. An average degree of 1 doesn't meet the hypothesis for any valid k , so the theorem simply gives no information (it doesn't claim the conclusion is false, just that this particular argument doesn't reach it).

Set B --- degree sequences

Q4. Is 2, 2, 2, 2, 2 (five vertices) graphical? If so, name the graph.

A. Sum = 10 (even, passes); max = $2 \leq n-1 = 4$ (passes). Havel–Hakimi: remove 2, subtract 1 from next 2: $\{2, 2, 2, 2, 2\} \rightarrow \{1, 1, 2, 2\} \rightarrow$ resorted $\{2, 1, 1, 2\}$... more carefully: $\{2, 2, 2, 2, 2\} \rightarrow \{1, 1, 2, 2\}$ resorted $\{2, 1, 1, 2\} \rightarrow$ resorted $\{2, 2, 1, 1\}$; remove 2, subtract from next 2: $\{1, 0, 1\}$ resorted $\{1, 1, 0\}$; remove 1, subtract from next 1: $\{0, 0\}$. All zero, no negatives — **graphical**. It's the 5-cycle C_5 (every vertex degree 2 is exactly the cycle-graph definition, Day 2).

Q5. Without running Havel–Hakimi, explain why 4, 4, 4, 4, 1 (five vertices) cannot be graphical.

A. Max entry is $4 = n-1 = 4$ — that alone passes the sandwich check, so this needs the sum check instead: sum = $4 + 4 + 4 + 4 + 1 = 17$, which is **odd** — violates the Handshaking Lemma. Not graphical.

Q6. Apply one step of Havel–Hakimi to $s = 4, 3, 3, 2, 2, 2$ and state the resulting sequence s_1 (resorted).

A. Remove the 4; subtract 1 from the next 4 entries ($3, 3, 2, 2 \rightarrow 2, 2, 1, 1$); leave the last entry (2) untouched.

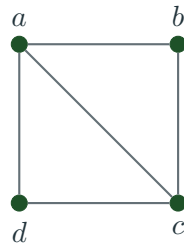
Raw result: $\{2, 2, 1, 1, 2\}$. Resorted: $s_1 = \{2, 2, 2, 1, 1\}$.

Just say “Day 6” whenever you’re ready for the next set (pages 102 onward, Paths & Circuits — Introduction).

CS-5136: Graph Theory

Paths & Circuits — Introduction (Part 1)

*pages 103–114 · walks, paths, distance,
concatenation & reversal, and two theorems on
connectedness — fully verified against the slides*



Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
this deck stays entirely within that source — no other book required
this section is large enough to need two parts — Day 7 continues at page 115

1 Walks and Paths: Getting the Language Right

Definition 1.1 (Walk)

A **walk** is a finite alternating sequence of vertices and edges, starting and ending with vertices, in which each edge is incident with the vertex immediately before it and the vertex immediately after it.

A wording problem worth fixing

Your slides add one more clause to this definition: “*and no edge appears more than once.*” That clause shouldn’t be there, and keeping it creates a real contradiction two slides later:

- The **Length** definition (Section 1.3 below) explicitly says a walk’s length counts edges “*including multiple occurrences of an edge.*” That sentence is meaningless if walks already can’t repeat edges.
- Your own slides separately define a **trail** as “a walk in which no edge appears more than once.” If ordinary walks already had that property, trails would be exactly the same thing as walks — a pointless definition.

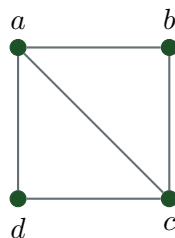
The fix: a general **walk** may repeat both vertices and edges freely. Restricting to *no repeated edges* gives a **trail**. Restricting further to *no repeated vertices* gives a **path** (and a path automatically has no repeated edges either — see the homework proposition below). So:

$$\text{path} \subseteq \text{trail} \subseteq \text{walk}.$$

Definitions 1.2

The vertices a walk begins and ends at are its **terminal vertices**. A walk that starts and ends at the *same* vertex is **closed**; otherwise it’s **open**. A **trail** is a walk with no repeated edge. An open walk with no repeated **vertex** is a **path**.

EXAMPLE — Walk vs. trail vs. path on the same graph



- **A walk that is not a trail:** a, b, c, b, a — uses edge ab twice and edge bc twice. Repeats edges, so it’s only a walk.
- **A trail that is not a path:** a, b, c, a, d — uses ab, bc, ca, ad , four *distinct* edges, but revisits vertex a . Repeats a vertex, so it’s a trail but not a path.
- **A path:** a, b, c, d — uses ab, bc, cd , all distinct vertices, all distinct edges.

Definitions 1.3 (Reachability & Easy Facts)

A vertex y is **reachable** from x if there’s a walk starting at x and ending at y . Some immediate consequences of the definitions above: a path cannot cross itself (no repeated vertex, by definition); a self-loop

can appear in a walk but never in a path (a loop's two ends are the same vertex, which would be a repeat); a path is itself a subgraph of G ; and in the subgraph formed by a path, every internal vertex has degree exactly 2 (one edge in, one edge out) while the two terminal vertices have degree exactly 1.

Proposition (posed without proof; due next class)

Let G be a simple graph and w a path in G . Then the edges of w are all distinct.

Proof. Write $w = (a_0, a_1, \dots, a_k)$, so its edges are $a_0a_1, a_1a_2, \dots, a_{k-1}a_k$. Suppose, for contradiction, two of these edges were the same: $a_ia_{i+1} = a_ja_{j+1}$ for some $i < j$. Since G is *simple*, there is at most one edge between any two vertices — so the edge a_ia_{i+1} can only mean the specific vertex pair $\{a_i, a_{i+1}\}$. For a different edge a_ja_{j+1} to be the exact same edge, it must be the exact same pair: either $(a_j = a_i$ and $a_{j+1} = a_{i+1})$ or $(a_j = a_{i+1}$ and $a_{j+1} = a_i)$. Either way, some vertex among $\{a_i, a_{i+1}\}$ equals some vertex among $\{a_j, a_{j+1}\}$ with $i \neq j$ (and $i+1 \neq j+1$ since $i \neq j$) — meaning w repeats a vertex. But w is a **path**, and paths have no repeated vertices, by definition. Contradiction. So no two edges of w can coincide. ■

1.1 Length, distance, diameter

Definitions 1.4

The **length** of a walk is its number of edges (counting repeats, if any). The **distance** $d_G(u, v)$ between two vertices is the length of a *shortest* u – v path; such a shortest path is called a u – v **geodesic**. The **diameter** $\text{diam}(G)$ of a connected graph is the largest distance between any two of its vertices.

2 Building New Walks: Concatenation and Reversal

Two small, almost-obvious operations on walks turn out to be exactly the ingredients needed to prove something important a little later (Section 2.3) — so it’s worth stating them carefully now.

Proposition 2.1 (Concatenation)

If $a = (a_0, \dots, a_k)$ is a walk from u to v and $b = (b_0, \dots, b_\ell)$ is a walk from v to w (so $a_k = b_0 = v$), then splicing them together at the shared vertex v — written $a * b$ — gives a walk from u to w , of length $k + \ell$.

Nothing deep here: gluing two alternating vertex–edge sequences at their shared endpoint just gives one longer alternating sequence, and every edge is still incident to its correct neighbours (nothing about the middle join changes that). The only subtlety is not to double-list the shared vertex v — write it once, not twice.

Proposition 2.2 (Reversal)

If $a = (a_0, \dots, a_k)$ is a walk from u to v , then $\text{rev}(a) := (a_k, a_{k-1}, \dots, a_0)$ is a walk from v to u of the same length. If a is a path, so is $\text{rev}(a)$.

Reading a valid alternating sequence backwards is still a valid alternating sequence (each edge is still incident to the same two neighbours, just visited in the other order) — and reversing obviously can’t create a repeated vertex that wasn’t already there.

2.1 Why this matters: connectedness is an equivalence relation

Definition 2.3 (The connectedness relation)

Define a relation $\simeq_G$ on $V(G)$ by $u \simeq_G v$ if and only if there is a walk from u to v in G .

EXAMPLE — This is exactly where those two propositions get used

$\simeq_G$ is an **equivalence relation**:

- *Reflexive*: the trivial length-0 walk (v) goes from v to v , so $v \simeq_G v$.
- *Symmetric*: if $u \simeq_G v$ via some walk a , then Proposition 2.2 turns a into a walk $\text{rev}(a)$ from v to u , so $v \simeq_G u$.
- *Transitive*: if $u \simeq_G v$ (walk a) and $v \simeq_G w$ (walk b), Proposition 2.1 splices them into a walk $a * b$ from u to w , so $u \simeq_G w$.

Every equivalence relation partitions its set into equivalence classes — here, those classes are exactly the **connected components** of G . G itself is connected precisely when there’s only one class, i.e. $u \simeq_G v$ for every pair u, v . This is the fully rigorous version of “connected” promised back in Day 5.

3 A Path Is Just a Shortest Walk

Proposition 3.1

Let G be simple, $u, v \in V(G)$, and $a = (a_0, \dots, a_k)$ a walk from u to v that is *not* a path. Then there is a walk from u to v that is strictly shorter than a .

Proof.

1. Since a is not a path, it repeats some vertex: there exist $i < j$ with $a_i = a_j$.
2. Build a new sequence by keeping the first part of a up to a_i , then jumping straight to what comes *after* a_j (which is the same vertex as a_i , so nothing is lost by skipping directly there): $(a_0, \dots, a_i, a_{j+1}, \dots, a_k)$.
3. This is still a valid walk from u to v — the join at $a_i = a_j$ behaves exactly like the concatenation in Proposition 2.1.
4. Its length is $i + (k - j)$ (the first piece contributes i edges, the second contributes $k - j$). Since $i < j$, we get $i + (k - j) < j + (k - j) = k$ — strictly shorter than a 's length k .



EXAMPLE

Applied to the walk a, b, c, b, a from the example above ($k = 4$, repeated vertex: $a_0 = a = a_4$, or more usefully $a_1 = b = a_3$): cutting out the loop between the two b 's leaves just (a, b, a) ... but $a_0 = a_2$ too, so cut again: leaves (a) , a length-0 walk. Repeating Proposition 3.1 always ends at an honest path — here, the trivial one.

A different perspective on five questions and one idea

Your slides pose five questions and ask which is “stronger”: does a u - v walk exist; does a u - v *path* exist; find the shortest path; find a walk of exact length k ; find a *path* of exact length k . Proposition 3.1 is exactly why questions 1 and 2 are secretly the *same* question: any walk can be shortened down to a path with the same endpoints, so a walk exists if and only if a path does. The remaining questions are genuinely different in difficulty: shortest path is efficiently solvable (breadth-first search); a walk of exact length k is easy to check via matrix powers of the adjacency matrix (a later topic); but asking for a *path* of an exact length k is, in general, a much harder computational problem.

4 Two Theorems on Connectedness

Theorem 4.1

Let G have order 3 or more. If G contains two distinct vertices u, v such that both $G - u$ and $G - v$ are connected, then G itself is connected.

Proof. We show every pair of vertices $x, y \in V(G)$ has a walk between them (equivalently a path, by Proposition 3.1) — so G is connected.

1. **Case A:** $x \neq u$ and $y \neq u$. Then x, y are both vertices of $G - u$, and $G - u$ is connected — so there's an x - y path in $G - u$, which is also a path in G (since $G - u$ is a subgraph of G).
2. **Case B:** $x \neq v$ and $y \neq v$. Symmetrically, use $G - v$ instead.
3. **Is every pair covered by A or B?** A pair fails *both* only if $(x = u \text{ or } y = u)$ and $(x = v \text{ or } y = v)$. With $u \neq v$, the only way to satisfy both is $\{x, y\} = \{u, v\}$ exactly — every other pair falls under Case A or Case B (if say $x = u$ but $y \neq v$, then $x \neq v$ automatically since $u \neq v$, so Case B applies).
4. **Case C:** $\{x, y\} = \{u, v\}$. Since $|V(G)| \geq 3$, pick a third vertex $w \neq u, v$. $G - v$ is connected, so there's a u - w path P' in $G - v$ (hence in G). $G - u$ is connected, so there's a w - v path P'' in $G - u$ (hence in G). Concatenating P' and P'' (Proposition 2.1) gives a u - v walk in G , and by Proposition 3.1 this yields a u - v path in G .

Every pair of vertices is connected by a path, so G is connected. ■

EXAMPLE

C_4 (the 4-cycle p, q, r, s): removing *any* single vertex leaves a path on the remaining 3 vertices, still connected. So *every* pair u, v satisfies the theorem's hypothesis here — consistent with C_4 itself being connected.

Theorem 4.2 (an Ore-type condition)

Let G have order n . If $d(u) + d(v) \geq n - 1$ for *every* pair of nonadjacent vertices u, v , then G is connected and $\text{diam}(G) \leq 2$. The bound $n - 1$ cannot be weakened.

Proof (connectedness). Suppose, for contradiction, G is disconnected. Pick any vertex u in one component (say of size n_1) and any vertex v in a *different* component (so all remaining components together have $n - n_1$ vertices). u, v are automatically nonadjacent (different components share no edges). Now:

- u can only be adjacent to other vertices in *its own* component, so $d(u) \leq n_1 - 1$.
- v 's entire component (and everything else outside u 's) has at most $n - n_1$ vertices, so $d(v) \leq (n - n_1) - 1$.

Adding these: $d(u) + d(v) \leq (n_1 - 1) + (n - n_1 - 1) = n - 2 < n - 1$, contradicting the hypothesis that *every* nonadjacent pair sums to at least $n - 1$. So G is connected. (This argument works for *any* number of components of *any* sizes — no need to assume exactly two equal halves.) ■

Proof (diameter ≤ 2). Take any two vertices x, y . If adjacent, $d(x, y) = 1 \leq 2$. If not, the hypothesis gives $d(x) + d(y) \geq n - 1$. Suppose x, y had *no* common neighbour: then $N(x)$ and $N(y)$ are disjoint subsets of $V \setminus \{x, y\}$ (neither set contains x or y , since $x \not\sim y$), so $d(x) + d(y) = |N(x)| + |N(y)| \leq |V \setminus \{x, y\}| = n - 2 < n - 1$ — contradiction. So x, y share a common neighbour z , giving a length-2 path x, z, y : $d(x, y) \leq 2$. ■

EXAMPLE — Checking the theorem and its sharpness

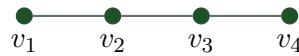
$G = K_4$ minus the edge ab (vertices a, b, c, d ; edges ac, ad, bc, bd, cd): the only nonadjacent pair is (a, b) , with $d(a) + d(b) = 2 + 2 = 4 \geq n - 1 = 3$. ✓ Indeed G is connected, and $\text{diam}(G) = 2$ (the a – b distance, via c or d).

Sharpness: two disjoint edges ab and cd ($n = 4$, disconnected). Every cross pair (e.g. a, c) has degree sum $1 + 1 = 2 = n - 2$ — one short of $n - 1 = 3$. This shows $n - 2$ is *not* enough to force connectivity: the theorem's threshold $n - 1$ is the best possible.

EXAMPLE — Resolving the homework: true or false?

“Every connected graph G of order 4 or more contains three distinct vertices u, v, w such that $G - u$, $G - v$, $G - w$ are all connected.”

False. Take the path $P_4 = v_1 - v_2 - v_3 - v_4$.



Removing an *endpoint* (v_1 or v_4) leaves a shorter path — still connected. But removing an *internal* vertex (v_2 or v_3) splits P_4 into two separate pieces — disconnected. So only **two** vertices (v_1, v_4) have the required property, never three, for any path of order ≥ 4 . This single counterexample disproves the statement.

5 Day 6 Cheat-Sheet

Recap

- $\text{path} \subseteq \text{trail} \subseteq \text{walk}$ (fewer repeats allowed as you go left). A walk may repeat vertices *and* edges freely.
- Length = edge count (repeats included); distance = shortest path length; diameter = largest distance in a connected graph.
- Concatenation and reversal of walks (Props 2.1–2.2) are exactly what’s needed to show $\simeq_G$ (“there’s a walk between them”) is an equivalence relation — its classes are the components of G .
- Any non-path walk can be shortened (cut out the repeated-vertex loop) — so walk-existence and path-existence are the same question.
- Two vertices whose removal each keep G connected $\Rightarrow G$ connected (order ≥ 3).
- $d(u) + d(v) \geq n - 1$ for every nonadjacent pair $\Rightarrow G$ connected, $\text{diam}(G) \leq 2$; $n - 1$ is the best possible bound.

6 Practice Questions

Set A — walks, paths, and the shortening argument

Q1. Classify a, b, c, a on the square-plus-diagonal graph (Section 1) as a walk, trail, and/or path.

A. Edges used: ab, bc, ca — three distinct edges (it’s a trail), and also three distinct vertices a, b, c visited before returning to the start... but it returns to a , so it’s closed, not open — and paths must be open. So: walk $\checkmark$, trail $\checkmark$, but not a path (it’s actually a cycle, previewed in the next chapter).

Q2. Using Proposition 3.1’s method, shorten the walk a, d, c, a, b on the square-plus-diagonal graph (Section 1) into a path.

A. Repeated vertex: a at position 0 and position 3. Cut: keep $(a_0) = a$, then jump to what follows position 3, i.e. $a_4 = b$: result (a, b) , length 1 — already a path (trivially).

Set B — connectedness theorems

Q3. Does Theorem 4.1 apply to a triangle K_3 ? (Careful: check the order requirement first.)

A. Yes — K_3 has order 3, which does meet “order 3 or more” (it’s easy to misread the requirement as excluding the boundary case, but it doesn’t). Removing any one vertex from K_3 leaves a single edge (still connected), so any two vertices u, v satisfy the hypothesis, correctly concluding K_3 is connected.

Q4. A graph has order 6. Two nonadjacent vertices u, v have $d(u) + d(v) = 4$. Does Theorem 4.2 guarantee anything here?

A. No. The theorem needs every nonadjacent pair to reach $n - 1 = 5$; a single pair with sum $4 < 5$ tells us nothing — the hypothesis must hold for all nonadjacent pairs simultaneously, not just one.

Q5. Find a connected graph of order 5 with exactly two vertices whose removal keeps it connected (like P_4 ’s endpoints, but order 5).

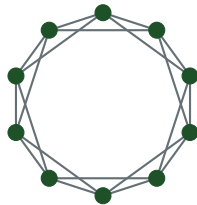
A. $P_5 = v_1-v_2-v_3-v_4-v_5$: removing either endpoint (v_1 or v_5) leaves P_4 (connected); removing any internal vertex (v_2, v_3, v_4) splits it into two pieces. Exactly two “safe” vertices, matching the pattern from the homework resolution above.

Just say “Day 7” to continue this same chapter (pages 115 onward: generating regular graphs, cycles & girth, and the four-cubes puzzle).

CS-5136: Graph Theory

Paths & Circuits — Introduction (Part 2) — Rebuilt

*pages 115–137 · every theorem built up
from a warm-up example, with every “if and only
if” proved in both directions, explicitly labelled*



Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
this is a from-scratch rebuild of Day 7, at a slower and more careful pace
this section is large — Day 8 continues with Euler Graphs

Before We Start: What Does “If and Only If” Mean?

Several theorems below have the shape “*G* has property *A* **if and only if** *G* has property *B*.” This single phrase is secretly making **two** separate promises, and a full proof has to keep both of them:

- **The “if” part** ($B \Rightarrow A$): *whenever* *B* is true, *A* is guaranteed to be true too. This direction lets you *conclude* *A* once you’ve checked *B*.
- **The “only if” part** ($A \Rightarrow B$): *A* can *only* happen when *B* is also true — equivalently, if *B* fails, *A* can’t possibly hold. This direction lets you go the *other* way: from knowing *A*, deduce *B*.

An everyday example: “*the ground is wet if and only if it rained.*” The “if” direction says: $\text{rain} \Rightarrow \text{wet ground}$. The “only if” direction says: $\text{wet ground} \Rightarrow \text{rain}$. Both directions here are actually a little suspicious in real life (a sprinkler could wet the ground without rain!) — exactly the kind of gap a mathematical proof isn’t allowed to leave. Every “if and only if” theorem below is proved by nailing down *both* arrows separately, with nothing assumed for free.

Not every theorem in this chapter is an “if and only if” though. Some only claim *one* direction. When that happens, we’ll say so explicitly, and — wherever it’s meaningful to ask — check whether the reverse direction is even *true*, rather than leave it a silent mystery. Sometimes it will genuinely hold; sometimes we’ll find a counterexample; and sometimes the reverse direction simply isn’t a sensible question to ask for that particular theorem (we’ll explain why, when that’s the case).

1 Generating Regular Graphs

Day 2 defined r -regular (every vertex has degree r) and showed K_n is one example. A natural question: for which r, n does an r -regular graph on n vertices exist at all — and can we always *build* one?

1.1 A warm-up before any formula

Does a 3-regular graph on 4 vertices exist? Every vertex would need degree 3 — but with only 4 vertices total, degree 3 means adjacent to *everyone else*. That's exactly K_4 : check it, every vertex has degree 3. So yes, for $(r, n) = (3, 4)$ it works.

Now try $(r, n) = (3, 5)$: can 5 vertices each have degree exactly 3? Total degree would be $5 \times 3 = 15$ — but the Handshaking Lemma (Day 2) says total degree must always be **even**, and 15 is odd. Impossible. Notice what made the difference: in the first case $n = 4$ was even; in the second, *both* $r = 3$ and $n = 5$ were odd. This warm-up is the entire theorem in miniature.

Theorem 1.1

Let r, n be integers with $0 \leq r \leq n - 1$. An r -regular graph of order n exists **if and only if** at least one of r, n is even.

Two directions, as always with an “if and only if”.

Proof (Direction 1: if a graph works, r or n must be even). If a graph is r -regular with n vertices, the sum of all degrees is $n \times r$. By the Handshaking Lemma (Day 2), this sum must be **even**. If both r and n were odd, $n \times r$ would be odd $\times$ odd = odd — impossible. So at least one of r, n must be even. ■

Proof (Direction 2: if r or n is even, a graph can actually be built). Arrange the n vertices $v_1, \dots, v_n$ evenly around a circle (indices taken mod n).

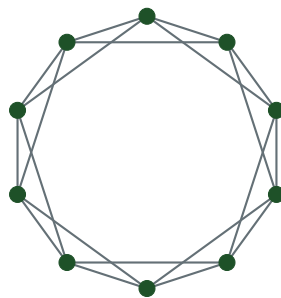
Case $r = 2k$ even. Join each v_i to the k nearest vertices clockwise *and* the k nearest counterclockwise: $v_{i+1}, \dots, v_{i+k}$ and $v_{i-1}, \dots, v_{i-k}$. Every vertex gains exactly $2k = r$ new neighbours this way (the construction is symmetric around the circle), so the result is r -regular. This requires $k \leq \frac{n-1}{2}$, i.e. $r \leq n - 1$ — exactly the given range.

Case $r = 2k + 1$ odd (so n must be even, say $n = 2\ell$). Build the same $2k$ -regular circle graph as above, then *additionally* join every v_i to its diametrically opposite vertex $v_{i+\ell}$. Since $n = 2\ell$, this pairs up all n vertices with no overlaps, adding exactly 1 to every vertex's degree — giving $2k + 1 = r$ -regular overall. ■

Naming

Graphs built this way are called **Harary graphs**, $H_{r,n}$, after Frank Harary.

EXAMPLE — $H_{4,10}$: a 4-regular graph on 10 vertices



$r = 4 = 2(2)$: each vertex joined to its 2 nearest neighbours each side.

(Building $H_{5,10}$ from this would add one more edge per vertex, to the diametrically opposite vertex,

giving 5-regular.)

1.2 Embedding any graph inside a regular one

Theorem 1.2

For every graph G and every integer $r \geq \Delta(G)$, there is an r -regular graph H containing G as an induced subgraph.

Is there a converse worth asking here? No — and here's why

This theorem's shape is “given G and r , here's how to build H ” — a construction, not a back-and-forth equivalence between two properties of the *same* object. There's no comparable reverse statement to test (asking “if H is r -regular and contains G as an induced subgraph, does $r \geq \Delta(G)$ hold?” is actually just true *by definition* — G 's own maximum degree can never exceed the degree of vertices in a bigger graph containing it — so it wouldn't teach us anything new to state as a separate “direction”). Not every theorem needs a converse check; this is one of the straightforward ones.

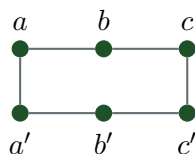
Proof. If G is already r -regular, take $H = G$. Otherwise $\delta(G) < r$ (some vertex has room to spare). The idea: take a second, identical copy of the whole graph, and connect every under-full vertex to its twin — this raises the *minimum* degree by exactly 1 without ever pushing anyone over r . Repeat until the minimum degree reaches r .

1. Let G' be a disjoint copy of G , with v'_i corresponding to v_i . Form $G_1 := G \cup G'$ plus, for every vertex v_i with $d_G(v_i) < r$, a new edge $v_i v'_i$.
2. **G is still an induced subgraph of G_1 :** none of the new edges run between two vertices of the original copy, so nothing changes among the original vertices.
3. **Every vertex's degree in G_1 is $d_G(v_i)$ or $d_G(v_i) + 1$,** and it's $+1$ exactly for vertices that had degree $< r$ before. Since $r \geq \Delta(G) \geq$ every degree, no vertex can be pushed past r (a vertex at exactly r gets no new edge, one below r becomes at most r).
4. **The minimum degree strictly increases:** every vertex that achieved the old minimum $\delta(G)$ satisfies $\delta(G) < r$ (since $r \geq \Delta(G) \geq \delta(G)$, with equality only in the already-regular case handled above) — so *all* of them gain $+1$. No vertex can end up below $\delta(G) + 1$ either (every vertex started at $\geq \delta(G)$ and either kept its value, which was then $\geq r > \delta(G)$, or gained $+1$). So $\delta(G_1) = \delta(G) + 1$.
5. Repeat this doubling on G_1 , then G_2 , and so on. Each round raises the minimum degree by 1 while capping the maximum at r . After $r - \delta(G)$ rounds, minimum degree $= r =$ maximum degree — every vertex has degree exactly r . Let H be this final graph.

■

EXAMPLE — A small self-contained check

Let $G = P_3 = a-b-c$ ($\Delta(G) = 2$, want $r = 2$; $\delta(G) = 1$ at a, c). One doubling round: copy a', b', c' ; since $d(a) = 1 < 2$ and $d(c) = 1 < 2$ but $d(b) = 2$ already, add edges aa' and cc' only.



This is the 6-cycle a, b, c, c', b', a — every vertex now has degree exactly 2. One round sufficed here since $r - \delta(G) = 2 - 1 = 1$. ✓ And $G = P_3$ sits inside it, induced, exactly as promised.

2 Cycles, Girth, and Components

Definitions 2.1

A closed walk $(v_1, \dots, v_k)$ with $v_1 = v_k$ is a **circuit**. If in addition $k \geq 3$ and $v_1, \dots, v_{k-1}$ are all distinct, it's a **cycle**. The **girth** of a graph is the length of its shortest cycle. A **component** is a maximal connected subgraph — equivalently (Day 6), one equivalence class of $\simeq_G$.

A typo to know about

Your slides spell this “*grith*” (in both the definition title and body). The correct term — the one you’ll find in every textbook and should use on an exam — is **girth**.

2.1 A warm-up before the connectedness theorem

Take a graph on $\{1, 2, 3, 4, 5\}$ with edges $12, 23$ (a small piece) and 45 (a separate piece) — clearly two components, $\{1, 2, 3\}$ and $\{4, 5\}$. Notice there is genuinely **no edge at all** running between the two groups $\{1, 2, 3\}$ and $\{4, 5\}$ — that’s exactly what makes them separate components in the first place, and it’s exactly the idea the theorem below turns into a general equivalence.

Theorem 2.2

G is disconnected **if and only if** $V(G)$ splits into two nonempty, disjoint sets V_1, V_2 with no edge running between them.

Proof (Direction 1: disconnected $\Rightarrow$ such a split exists). If G is disconnected, let V_1 be the vertices of one component and V_2 everything else (at least one other component exists, so V_2 is nonempty too). By definition of “component”, there’s no edge crossing between different components — so no edge runs between V_1 and V_2 . ■

Proof (Direction 2: such a split existing $\Rightarrow$ disconnected). Suppose V_1, V_2 is such a split. Take any $u \in V_1$ and $v \in V_2$. Could a walk go from u to v ? Every single edge of G has both its endpoints entirely inside V_1 or entirely inside V_2 (since no edge crosses between them) — so a walk starting in V_1 can only ever move to other vertices still inside V_1 , never escaping into V_2 . So $u \not\sim_G v$ (Day 6’s connectedness relation) — meaning G has more than one equivalence class, i.e. is disconnected. ■

2.2 Two more theorems

Theorem 2.3

A simple graph with n vertices and k components has at most $\frac{(n-k)(n-k+1)}{2}$ edges.

Not an iff — it’s a bound achieved by a specific construction

This theorem states a *maximum*, and (like Day 9’s K_n decomposition later this series) the natural thing to check isn’t a “converse” but whether the bound is actually *achievable*. We show both halves below: the bound itself, and a specific graph that hits it exactly.

Proof. Let the components have $n_1, \dots, n_k$ vertices, so $n_1 + \dots + n_k = n$. Within a single component of size n_i , at most $\binom{n_i}{2}$ edges are possible (Day 2, Theorem 2.12), so the total is at most $\sum_i \binom{n_i}{2} = \frac{1}{2}(\sum_i n_i^2 - n)$ (using $\sum n_i = n$). We now find the worst case — the split that makes this as large as possible.

Concentrate, don’t spread out. Fix the total $\sum n_i = n$ and the count k . Among all ways to split n into k positive parts, $\sum n_i^2$ is *largest* when the parts are as unequal as possible: pushing one part up by 1 while

pulling another down by 1 (keeping the sum fixed) changes $\sum n_i^2$ by $+2(\text{difference}) + 2 > 0$ whenever the part going up was already $\geq$ the part going down — so any spread-out configuration can be improved by concentrating further, until $k - 1$ parts are stuck at the minimum allowed value ($n_i = 1$, since a component needs at least one vertex) and everything else piles into one part, $n - (k - 1) = n - k + 1$.

Plugging in $n_1 = \dots = n_{k-1} = 1, n_k = n - k + 1$:

$$\sum_i n_i^2 = (k - 1)(1)^2 + (n - k + 1)^2.$$

So the maximum edge count is

$$\frac{1}{2} \left[(k - 1) + (n - k + 1)^2 - n \right].$$

Expand $(n - k + 1)^2 = (n - k)^2 + 2(n - k) + 1$ and simplify:

$$(k - 1) + (n - k)^2 + 2(n - k) + 1 - n = (n - k)^2 + 2(n - k) + k - n = (n - k)^2 + (n - k),$$

using $k - n = -(n - k)$. So the maximum is $\frac{1}{2} [(n - k)^2 + (n - k)] = \frac{(n - k)(n - k + 1)}{2}$, exactly the claimed bound — achieved by one big component of size $n - k + 1$ (as complete as possible) and $k - 1$ isolated vertices. ■

Theorem 2.4

If a graph (connected or not) has *exactly* two vertices of odd degree, there is a path joining them.

Not an iff — and no meaningful converse to ask

This theorem's hypothesis is a *count* (exactly two odd-degree vertices); its conclusion is about a *path existing between those two specific vertices*. Swapping the direction — “if a path joins some pair, must the graph have exactly two odd vertices?” — isn't even a sensible statement about the same situation (paths exist between countless pairs in a connected graph regardless of any degree parity). So there's genuinely nothing symmetric to check here — this is a clean one-directional fact.

Proof. Let v_1, v_2 be the two odd-degree vertices. If G is connected, done trivially. Otherwise, suppose for contradiction v_1, v_2 lie in *different* components C_1, C_2 . A vertex's degree only counts edges within its own component, so C_1 contains exactly one odd-degree vertex (v_1 — every other vertex of G , including everyone else in C_1 , has even degree) and likewise C_2 has exactly one (v_2). But C_1 is itself a graph, and by the Handshaking Lemma's corollary (Day 2), *every* graph has an **even** number of odd-degree vertices — yet C_1 has exactly 1, an odd count. Contradiction. So v_1, v_2 must share a component, hence (that component being connected) a path joins them. ■

3 The Four-Cubes Puzzle

This next part is a solved puzzle, not a theorem — so there’s no “if and only if” or converse to track here, just a fun, fully verified application of everything above.

The puzzle

Given four cubes whose faces are colored red, blue, green, yellow, can they be stacked into a 4×1 tower so that all four colors appear exactly once on *each* of the tower’s four long sides (left, right, front, back)?

EXAMPLE — The four cubes (unfolded)

Cube 1	Cube 2	Cube 3	Cube 4
T:R, Bo:R	T:R, Bo:Y	T:G, Bo:G	T:B, Bo:Y
L:R, F:Y, R:G, B:B	L:R, F:Y, R:B, B:G	L:B, F:B, R:R, B:Y	L:G, F:Y, R:R, B:G

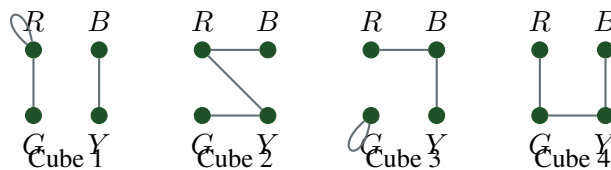
(T/Bo/L/F/R/B = top/bottom/left/front/right/back on the original, unrotated net.)

3.1 Turning it into graph theory

The encoding (your slides’ method)

For each cube, build a graph on 4 vertices $\{R, B, G, Y\}$: join two colors by an edge exactly when they sit on *opposite* faces of that cube. Opposite-face pairs are: (Top, Bottom), (Left, Right), and (Front, Back).

EXAMPLE — The four cube-graphs — verified against the slides



Cube 1: loop at R (Top=Bottom= R), R - G (Left-Right), B - Y (Front-Back). **Cube 2:** R - Y (T-B), R - B (L-R), G - Y (F-B). **Cube 3:** loop at G (T-B), B - R (L-R), B - Y (F-B). **Cube 4:** B - Y (T-B), G - R (L-R), Y - G (F-B).

What a solution needs

Take the union of all four cube-graphs (every edge labelled by which cube it came from — 12 edges total, since each cube gives 3). A stacking that solves the puzzle corresponds to finding **two** edge-disjoint subgraphs G_1 (the Left–Right assignment) and G_2 (the Front–Back assignment), each containing *exactly one edge from every cube*, and each with **every vertex at degree exactly 2** — so that, reading off G_1 ’s edges, each colour appears exactly once as “Left” and once as “Right” across the four cubes (and likewise Front/Back from G_2).

EXAMPLE — A verified solution

$$G_1 : \{R-G, B-R, Y-B, G-Y\} \quad (\text{cubes 1, 2, 3, 4 respectively}),$$

$$G_2 : \{Y-B, G-Y, B-R, R-G\} \quad (\text{cubes 1, 2, 3, 4}).$$

Both are the same 4-cycle $R-G-Y-B-R$, just with different cubes supplying each edge — and for every cube, its G_1 -edge and G_2 -edge are different, so the two are edge-disjoint *per cube*, as required. Every colour has degree 2 in each, exactly as needed. Reading this off directly gives the Left/Right and Front/Back face for every cube in the stack.

EXAMPLE — The verified solution in full

	Left	Front	Right	Back
Cube 1 (bottom)	<i>R</i>	<i>Y</i>	<i>G</i>	<i>B</i>
Cube 2	<i>B</i>	<i>G</i>	<i>R</i>	<i>Y</i>
Cube 3	<i>Y</i>	<i>B</i>	<i>B</i>	<i>R</i>
Cube 4 (top)	<i>G</i>	<i>R</i>	<i>Y</i>	<i>G</i>

Reading down each column: Left = *R, B, Y, G*; Front = *Y, G, B, R*; Right = *G, R, B, Y*; Back = *B, Y, R, G* — **every column is a permutation of $\{R, B, G, Y\}$** , so yes, the four cubes *can* be stacked with all four colours on every side. Puzzle solved.

4 Closing the Chapter's Last Homework

Proposition (originally posed as homework)

If v is a vertex of G and $G - v$ is 3-regular, then G has odd order.

Proof. Let $G - v$ have m vertices, so $|V(G)| = m + 1$. Since $G - v$ is 3-regular, the sum of its degrees is $3m$. By the Handshaking Lemma, this sum must be even — and since 3 is odd, $3m$ is even only when m itself is even. So m is even, making $|V(G)| = m + 1$ **odd**. ■

EXAMPLE — Checking the converse honestly: does it fail?

Does *odd order* $\Rightarrow$ *$G - v$ is 3-regular for some v* ? Take G = a single triangle (order 3, odd). For *any* choice of v , $G - v$ is just a single edge on the remaining 2 vertices — that's 1-regular, not 3-regular. So odd order alone guarantees nothing about any $G - v$ being 3-regular — the converse fails immediately, and rather dramatically (it's not even close; $G - v$ here has the wrong number of vertices to ever *be* 3-regular in the first place, since 3-regular needs at least 4 vertices).

5 Day 7 Cheat-Sheet

Recap: which theorems are “iff” and which aren’t

- **Iff (both directions proved):** r -regular graph of order n exists $\iff r$ or n is even; G disconnected $\iff V(G)$ splits into two parts with no edge between them.
- **One direction, no meaningful converse to ask:** the embedding theorem (any G fits inside some r -regular H); the max-edges-in-a-disconnected-graph bound (achieved by a specific construction, not a converse); the odd-degree-pair path theorem.
- **One direction, converse checked and found dramatically false:** the closing homework ($G - v$ 3-regular $\Rightarrow$ odd order; a triangle shows odd order alone guarantees nothing back).
- Harary graphs $H_{r,n}$: circle + k nearest each side, plus opposite-vertex edges if r is odd.
- Cycle: closed walk, length ≥ 3 , no repeated vertex except the shared endpoint. Girth = shortest cycle length (not “grith”!).
- Instant Insanity: encode each cube as a graph on $\{R, B, G, Y\}$ via opposite faces; find two edge-disjoint (per cube), 2-regular subgraphs using one edge per cube each.

6 Practice Questions

Set A — regular graphs

Q1. Does a 3-regular graph on 7 vertices exist? On 8 vertices?

A. On 7: both $r = 3$ and $n = 7$ are odd — no such graph exists. On 8: $n = 8$ is even, so it exists (e.g. the cube graph Q_3 , or a Harary graph $H_{3,8}$).

Q2. In $H_{4,10}$, what is $d(v_1)$, and which vertices is v_1 adjacent to?

A. $d(v_1) = 4$. Adjacent to the 2 nearest each side: v_2, v_3 (clockwise) and v_{10}, v_9 (counterclockwise, i.e. $v_{1-1} = v_{10}$ and $v_{1-2} = v_9 \bmod 10$).

Set B — cycles, components, and the two theorems

Q3. What is the girth of K_4 ? Of a tree (any tree)?

A. K_4 : girth 3 (any three vertices form a triangle). A tree has no cycles at all, so its girth is conventionally undefined (or ∞).

Q4. A simple graph has 10 vertices and 3 components. What is the maximum possible number of edges?

A. $\frac{(n-k)(n-k+1)}{2} = \frac{(10-3)(10-3+1)}{2} = \frac{7 \times 8}{2} = 28$ — achieved by K_8 plus two isolated vertices.

Q5. A graph has degree sequence 3, 3, 2, 2, 1, 1 split across two components of 3 vertices each. Can Theorem 2.4 apply?

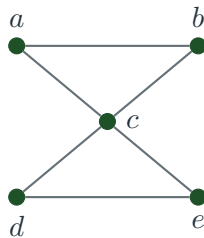
A. Theorem 2.4 needs exactly two odd-degree vertices in the whole graph. Here the odd-degree vertices are the two of degree 3 and the two of degree 1 — that’s four, not two, so the theorem doesn’t apply.

Just say “Day 8” whenever you’re ready for the next chapter (pages 138 onward, Euler graphs).

CS-5136: Graph Theory

Euler Graphs — Rebuilt From First Principles

*pages 138–160 · every theorem built up
slowly with a warm-up example first, and every
“if and only if” proved in both directions, explicitly*



Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
this is a from-scratch rebuild of Day 8, at a slower and more careful pace

Before We Start: What Does “If and Only If” Mean?

Almost every theorem in this document has the shape “*G* has property *A* **if and only if** *G* has property *B*.” This one phrase is secretly making **two** separate promises, and a full proof has to keep both of them:

- **The “if” part** ($B \Rightarrow A$): *whenever* *B* is true, *A* is guaranteed to be true too. This direction lets you *conclude* *A* once you’ve checked *B*.
- **The “only if” part** ($A \Rightarrow B$): *A* can *only* happen when *B* is also true — equivalently, if *B* fails, *A* can’t possibly hold. This direction lets you go the *other* way: from knowing *A*, deduce *B*.

A everyday example: “*the ground is wet if and only if it rained.*” The “if” direction says: $\text{rain} \Rightarrow \text{wet ground}$ (rain guarantees wetness). The “only if” direction says: $\text{wet ground} \Rightarrow \text{rain}$ (nothing else could have made it wet). Both directions here are actually a little suspicious in real life (a sprinkler could wet the ground without rain!) — which is exactly the kind of gap a mathematical proof isn’t allowed to leave. Every “if and only if” theorem below is proved by nailing down *both* arrows separately, with nothing assumed for free.

Not every theorem in this chapter is an “if and only if” though — some (like Ore’s theorem, coming in Day 9) only claim *one* direction. When that happens, we’ll say so explicitly, and check whether the reverse direction is even *true* (often it isn’t — we’ll show a counterexample rather than leave it a mystery).

1 Where This Chapter Comes From

In 1736, the city of Königsberg (now Kaliningrad) straddled the river Pregel, with two islands connected to the mainland and each other by seven bridges. Locals wondered: could you walk a route crossing every bridge exactly once? Leonhard Euler modelled the four land masses as vertices and the seven bridges as edges, and showed the answer is *no* — laying the foundation of graph theory in the process. (Your slides open with a poem about this by William T. Tutte; the original is worth reading in full elsewhere, but the gist is: Königsberg's citizens tried and failed, again and again, to walk all seven bridges without repeating one.) Euler's reasoning is exactly the theorem this chapter opens with.

2 Euler Graphs

2.1 Building the definitions, one small step at a time

Step 1: what is a closed walk that never repeats an edge?

Take any graph. Start at some vertex, and walk along edges, never using the *same edge twice*, until you arrive back where you started, having used up *every single edge* in the graph exactly once. If such a walk exists, call it an **Euler circuit**, and call the graph an **Euler graph**.

EXAMPLE — A tiny picture before any theory

Take a square with both diagonals: vertices a, b, c, d , edges ab, bc, cd, da (the square) plus ac, bd (the diagonals) — 6 edges total. Try: $a \rightarrow b \rightarrow c \rightarrow a \rightarrow d \rightarrow c \dots$ wait, that reuses vertex c 's connections awkwardly and misses bd . After some trial and error you'd find a, b, d, c, a using ab, bd, dc, ca — only 4 of the 6 edges, missing bc and ad . In fact, no matter how you try, you **cannot** walk all 6 edges of this graph in one closed loop without repeating an edge. Why not, in general (not just this one example)? That's exactly what the theorem below answers.

“Connected” — with a small asterisk

Your slides note: *an Euler graph must be connected, but can have isolated vertices*. This sounds contradictory until you read it precisely: the Euler circuit only has to cover every *edge* — so the vertices that carry edges must form one connected piece, but a vertex with *no* edges at all is simply never visited, and doesn't break anything. In short: the *non-isolated* part of the graph must be connected.

2.2 The main theorem, built up slowly

Before the general statement, notice something about the failed example above. Count the degree of each vertex in the square-plus-diagonals graph: a touches ab, ad, ac — degree 3. Same for b, c, d by symmetry: every vertex has degree 3. **Odd**. Now compare to a graph that *does* work: the bowtie (two triangles sharing vertex c) from before — degrees 2, 2, 4, 2, 2, all **even**. This pattern (odd degrees break it, even degrees fix it) is not a coincidence; it's the entire theorem.

Theorem 1.1 (Euler's theorem)

A connected graph G is an Euler graph **if and only if** every vertex of G has even degree.

This is an “if and only if”, so (per the box above) we owe you two separate proofs: one showing *Euler graph* $\Rightarrow$ *all degrees even*, and one showing *all degrees even* $\Rightarrow$ *Euler graph*. Neither direction is allowed to borrow from the other.

Proof (Direction 1: if G is an Euler graph, every degree is even). Suppose G has an Euler circuit. Pick any vertex v , and watch what the circuit does every time it visits v : it arrives along one edge and leaves along a *different* edge (it can't reuse an edge). So every visit to v uses up exactly 2 of v 's edges, paired together as “arrival” and “departure”. Since the circuit is closed and uses *every* edge of the whole graph exactly once, every one of v 's edges gets used during some visit — and every visit pairs up exactly 2 of them. (The very first edge leaving the start vertex and the very last edge arriving back count as a pair too, since the circuit is closed.) With all of v 's edges perfectly paired into twos, v 's total degree must be even. This holds for every vertex v , so every degree is even. ■

Proof (Direction 2: if G is connected and every degree is even, G is an Euler graph). This direction has to actually *build* the circuit, so it takes a little longer. We'll do it in two stages.

Stage A — one closed trail always exists. Start at any vertex v_0 and keep walking along edges you haven't

used yet, for as long as possible. Claim: this walk can only get stuck back at v_0 , never anywhere else. Why: at any *other* vertex u , every time you arrive you use up 1 edge; since $\deg(u)$ is even, the edges at u can always be perfectly paired into arrival/departure twos, so there's always a fresh edge to leave by — you can never truly run out of options at u . So the walk is only forced to stop once it returns to v_0 with no unused edges left there. This gives one closed trail C (possibly not using every edge yet).

Stage B — stitch in whatever's missing. If C already used every edge, we're done. Otherwise, some edges are still unused. Since G is connected, at least one unused edge must touch some vertex w that C *does* pass through (if no unused edge touched any vertex of C , the unused edges would be completely cut off from C 's vertices, contradicting connectedness). Removing C 's edges from G leaves every vertex with even degree still (each vertex loses degree in pairs, exactly as in Stage A's argument) — so we can repeat Stage A *starting from w* , using only the remaining unused edges, to get another closed trail C' through w . Now splice C' into C : walk C until you reach w , take the entire detour around C' (ending back at w), then continue along the rest of C . This gives one longer closed trail using every edge of C and C' together. Repeat this stitching process — each round uses up more of the (finitely many) edges, so it must eventually stop, at which point every edge has been incorporated into one single closed trail: an Euler circuit. ■

EXAMPLE — Checking both directions on our two running examples

Square-plus-diagonals (the graph that failed): every degree is 3, odd. Direction 1 (in its contrapositive form: odd degree somewhere $\Rightarrow$ not an Euler graph) instantly explains why no Euler circuit could ever be found, no matter how hard we tried above. **The bowtie**: degrees 2, 2, 4, 2, 2, all even, and connected. Direction 2 guarantees an Euler circuit exists — and indeed a, b, c, d, e, c, a is one, using all 6 edges.

2.3 A closed trail is always a stack of cycles

This next fact is *not* an “if and only if” — it's a one-directional building block we'll reuse constantly.

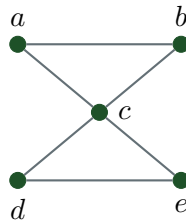
Theorem 1.2

Every nontrivial closed trail (tour) is composed of edge-disjoint cycles.

Proof. By induction on the trail's length ℓ (i.e. we prove it for short trails first, then show a longer trail can always be reduced to a shorter, already-handled case).

Base case ($\ell = 1$). A closed trail of length 1 is a single edge from a vertex to itself — a self-loop, which is already a (degenerate) cycle all on its own. Nothing more to prove here.

Inductive step. Suppose the claim is already known for every closed trail of length $\leq n$. Let t be a closed trail of length $n + 1$. If t never revisits any vertex (except of course at the very last step, closing the loop), then t is already a single cycle — done. Otherwise, t revisits some vertex v partway through. The stretch of t between v 's two visits is itself a closed trail — call it c (if c itself revisits a vertex, shorten it first via Day 6's shortening trick, so assume c is a genuine cycle). Deleting c 's edges from t leaves a shorter closed trail t' , of length $\leq n - 1 \leq n$ (it lost at least c 's ≥ 3 edges, or ≥ 1 if c was a loop). By the inductive hypothesis, t' already decomposes into edge-disjoint cycles; adding c (which shares no edge with t' , since we removed precisely c 's edges and nothing else) completes the decomposition of t . ■

EXAMPLE — The “bowtie” — two triangles at a hinge

Degrees: a, b, d, e all degree 2; c has degree 4. All even, so this is an Euler graph. The tour a, b, c, d, e, c, a revisits c midway — splitting it exactly into the two edge-disjoint triangles $\{a, b, c\}$ and $\{c, d, e\}$, matching Theorem 1.2 precisely.

2.4 Two fun applications

EXAMPLE — The Star of David

A hexagram drawn as 6 outer points and 6 inner hexagon corners: each outer point connects to 2 inner corners (degree 2), and each inner corner connects to 2 neighbouring inner corners *and* 2 outer points (degree 4). Every degree is even, and the figure is connected — so by Theorem 1.1’s Direction 2, it *can* be drawn without lifting your pencil or retracing a line. (“Mohammed’s scimitar”, the companion puzzle on the same slide, is a different specific figure with the same underlying property — check any figure this way: count degrees, see if they’re all even.)

EXAMPLE — Resolving the homework: the join $G_1 + G_2$

Your slides define the **join**: $G = G_1 + G_2$ takes two vertex-disjoint graphs and adds *every* possible edge between $V(G_1)$ and $V(G_2)$, on top of each graph’s own edges.

First, a simple fact: $G_1 + G_2$ is *always* connected, regardless of whether G_1 or G_2 themselves were — since every single vertex of G_1 has a direct edge to every single vertex of G_2 , there’s no way to end up in two separate pieces.

Now, when is it Eulerian? Each vertex $v \in G_1$ gains exactly $|V(G_2)|$ new edges (one to every vertex of G_2), so its new degree is $\deg_{G_1}(v) + |V(G_2)|$. For this to be *even*, $\deg_{G_1}(v)$ needs the *same parity* (both odd, or both even) as $|V(G_2)|$. Symmetrically for every vertex of G_2 . So, since we already know $G_1 + G_2$ is always connected, Theorem 1.1’s Direction 2 tells us:

$$G_1 + G_2 \text{ is Eulerian} \iff \begin{cases} \text{every degree in } G_1 \equiv |V(G_2)| \pmod{2}, \text{ and} \\ \text{every degree in } G_2 \equiv |V(G_1)| \pmod{2}. \end{cases}$$

Check: $G_1 = G_2 =$ two isolated vertices each: every degree in G_1 is 0, and $|V(G_2)| = 2$ — same parity (both even) ✓. Indeed $G_1 + G_2 = C_4$, which is Eulerian. Contrast $G_1 = G_2 = K_1$ (single vertices, degree 0) with $|V(G_2)| = 1$: 0 is even, 1 is odd — different parity, so not Eulerian. Indeed $G_1 + G_2 = K_2$, a single edge, each vertex odd-degree.

3 Unicursal Graphs: When You Can't Quite Close the Loop

3.1 Motivation and definition

What if a graph has *almost* every degree even, except for two stubborn vertices? Intuitively, you should be able to walk every edge exactly once, just not return to where you started — you'd have to finish at that other odd vertex instead. Let's make this precise.

Definition 2.1 (Unicursal graph & Euler trail)

A graph with an *open* walk (starts and ends at **different** vertices) that traverses every edge exactly once is **unicursal**; that walk is a **unicursal line** or **Euler trail**.

EXAMPLE — A warm-up before the general theorem

Take a single path $a - b - c - d$ (3 edges). Degrees: $a = 1, b = 2, c = 2, d = 1$. Two odd-degree vertices (a, d), the rest even. Sure enough, the walk a, b, c, d (just walking straight along it) is an Euler trail, ending exactly at the *other* odd vertex from where it started. This already hints at the pattern: unicursal $\iff$ exactly two odd vertices.

Theorem 2.2

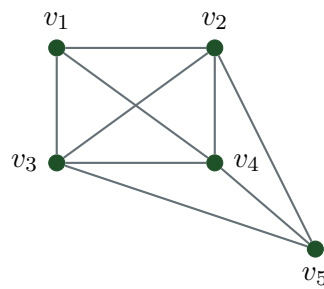
A connected graph G is unicursal **if and only if** it has exactly two vertices of odd degree.

Again, two directions to prove separately.

Proof (Direction 1: unicursal $\Rightarrow$ exactly two odd vertices). Suppose u -to- v is an Euler trail of G . Exactly as in Theorem 1.1's Direction 1, every vertex the trail merely *passes through* (not an endpoint) has its edges perfectly paired up by consecutive arrival/departure — even degree. The two *endpoints* are different: u only ever *departs* at the very start (one unpaired edge, since it never arrives there again — if it did, that would be a revisit partway through, still fine, but the crucial unpaired edge is specifically the very first departure with nothing before it to pair against); similarly v only ever *arrives* at the very end with nothing after it to pair against. So u, v each have one leftover unpaired edge — odd degree. Since $u \neq v$ (an open trail has different endpoints, by definition), that's exactly two odd-degree vertices, and (by the pairing argument) nobody else qualifies. ■

Proof (Direction 2: exactly two odd vertices $\Rightarrow$ unicursal). Suppose G is connected with exactly two odd-degree vertices u, v . Here's the trick: temporarily add one new edge uv to G (call the result H ; if uv already existed, just imagine a second parallel copy). Every vertex keeps its old degree in H *except* u, v , which each gain exactly 1 — turning their odd degrees even. So now *every* vertex of H has even degree, and H is connected (we only added an edge, never removed the connections G already had) — by Theorem 1.1's Direction 2, H has an Euler circuit.

That circuit must use our added edge uv somewhere (it uses every edge of H , and uv is one of them). Delete just that one edge from the circuit: a closed loop with one edge removed becomes a single open trail, running from u to v (or v to u — same thing), and using every *other* edge of H — which is exactly every edge of the original G . So G has an Euler trail, and by Direction 1 (just proved), its endpoints are forced to be exactly the two odd vertices u, v . ■

EXAMPLE — Verified example

Degrees: $d(v_1) = 3$, $d(v_2) = 4$, $d(v_3) = 4$, $d(v_4) = 4$, $d(v_5) = 3$ — exactly two odd vertices, v_1, v_5 . So G is unicursal with Euler trail endpoints v_1, v_5 : e.g. $v_1, v_2, v_3, v_1, v_4, v_2, v_5, v_3, v_4, v_5$ uses all 9 edges once (verified by computer — every edge appears exactly once, and no edge outside the graph is ever used). Adding edge $v_1 v_5$ (the construction's H) makes both degree 4 — fully Eulerian, exactly as Direction 2's proof describes.

4 Fleury’s Algorithm: Finding the Tour

Euler’s theorem says *when* a tour exists; it doesn’t say how to *find* one without accidentally stranding yourself partway through. Fleury’s algorithm gives a simple rule for building one by hand.

Fleury’s Algorithm

Given an Euler graph G : pick any starting vertex. Repeatedly walk along an edge incident to your current vertex and delete it — **except** never choose an edge whose removal would disconnect the remaining (non-isolated) graph, *unless* it’s your only option. Stop when no edges remain.

Theorem 3.1

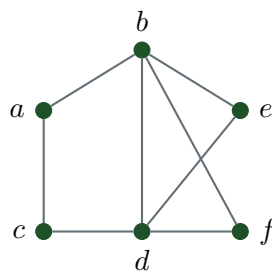
The walk W returned by Fleury’s Algorithm on an Euler graph G is an Euler tour of G .

This theorem is *not* an “if and only if” — it only claims one direction: *follow the rule* $\Rightarrow$ *you get a tour*. (We’ll come back at the end of this section to ask, honestly, whether the reverse question — does every actual Euler tour secretly follow this rule? — also holds.)

Proof. W is closed. The algorithm only stops when the current vertex has no remaining edges. Every other vertex visited along the way has even degree remaining at each pass (mirroring Theorem 1.1’s pairing argument: each visit pairs off two edges), so the walk can only run out of options exactly back at the start — otherwise it could always continue, contradicting that it stopped. So W is a closed trail.

W uses every edge. Suppose not: some edges remain unused when W ends. Since G is connected, some unused edge touches a vertex x that W *did* visit. At the moment W passed through x , the bridge-avoidance rule would only have permitted skipping that unused edge if taking it disconnected the *other* remaining unused edges from x ’s side — but W never returns to x again after this point (it’s already moved on for good), so those unused edges become permanently unreachable, contradicting that $G - \{\text{used edges}\}$ must stay reachable from wherever the walk still needs to go. This is exactly what the bridge rule is designed to prevent: it never strands a piece of the graph that still has edges in it. ■

EXAMPLE — Worked example from start to finish



Degrees: $a=2, b=4, c=2, d=4, e=2, f=2$ — all even, Euler graph. Running Fleury’s algorithm from a (verified in full, no bridge ever forced early):

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow b \rightarrow f \rightarrow d \rightarrow c \rightarrow a,$$

using all 8 edges exactly once and returning to a .

Does the reverse hold? Checking honestly

Could an Euler tour exist that *doesn’t* follow Fleury’s rule at some step — i.e. takes a bad disconnecting edge and gets away with it? Think it through: if a tour ever took an edge that cut off some remaining

unused edges from wherever it currently stands, it could *never* come back to use them (that piece is now unreachable from the current position for the rest of the walk) — so the tour would end without using every edge, meaning it **wasn't actually a valid Euler tour** in the first place. So in fact: any walk that *is* a genuine Euler tour must have avoided every disconnecting choice throughout, automatically. The reverse direction holds too — Fleury's rule isn't just one way to succeed, it's a rule every success story secretly followed all along.

5 A Generalization

Theorem 4.1

A connected graph G with exactly $2k$ odd-degree vertices splits into exactly k edge-disjoint subgraphs, together covering every edge of G , each of which is unicursal.

Like Theorem 3.1, this is a one-directional statement (a construction, not an equivalence) — there’s no natural “converse” to ask here, since the statement is already describing an if-then relationship between a count ($2k$ odd vertices) and a construction (k pieces), not a back-and-forth equivalence between two properties of the *same* kind.

Proof. Label the odd-degree vertices $v_1, \dots, v_k, w_1, \dots, w_k$ (any pairing-up of the $2k$ of them). Add k new edges $v_1w_1, v_2w_2, \dots, v_kw_k$. Each v_i, w_i gains exactly 1 to its degree, turning odd to even; nobody else changes. So the new graph is connected with all-even degree: Eulerian (Theorem 1.1), with some Euler tour ρ .

Now delete the k added edges from ρ , one at a time. Removing a single edge from a closed trail breaks it into one open trail (cut at that edge); removing a second (now on that open trail, or later if it landed on a separately-cut piece) splits further. After removing all k added edges, ρ has been cut into exactly k open pieces (each cut increases the piece-count by exactly 1, starting from 1 closed trail). Each piece is a walk using only original edges of G , with no repeated edge (inherited from ρ) — a unicursal line. Together they use every edge of ρ except the k added ones, i.e. exactly every edge of G , and no edge is shared between two pieces (each original edge appeared exactly once in ρ , hence in exactly one piece). ■

6 Arbitrarily Traceable Graphs

A cautionary example

On the bowtie graph, start an Euler tour greedily: $a \rightarrow b \rightarrow c$. From c , four edges remain (ca is one of them, plus cd, ce, de). Choosing $c \rightarrow a$ seems fine — it's not yet forced — but it's a trap: after $a \rightarrow b \rightarrow c \rightarrow a$, you're stuck at a with no edges left there, while cd, ce, de remain completely untouched and unreachable. The naive choice violated Fleury's bridge-avoiding rule (removing ca at that point *does* disconnect the still-unvisited triangle $\{c, d, e\}$ from a), even though it wasn't obviously a “bridge” in the usual single-edge sense.

Definition 5.1 (Arbitrarily traceable graph)

G is **arbitrarily traceable from** v if *every* walk starting at v that never reuses an edge can always be extended to a full Euler tour — i.e. you can never paint yourself into a corner, no matter which unused edge you pick at each step.

Claim (posed as a question in your slides)

G is arbitrarily traceable from v **if and only if** every cycle of G passes through v .

Being honest about what we can fully prove here

A complete, fully general proof of both directions of this claim needs machinery (about “blocks” or biconnected pieces of a graph) that goes beyond what this course has built up to this point. What we *can* do — and what genuinely explains *why* the claim is true — is verify both directions completely and rigorously on our concrete bowtie example, checking every single case by hand rather than waving at the general pattern.

EXAMPLE — Direction 1 verified completely: c works

Every cycle of the bowtie is either $\{a, b, c\}$ or $\{c, d, e\}$ — and both pass through c . Start at c ; there are 4 edges to choose from first. **Case ca :** continue c, a, b, c (forced, only path back), now at c again with cd, ce, de left — continue c, d, e, c , done, full tour. **Case cb :** symmetric, tour completes the same way. **Case cd :** continue c, d, e, c (forced back to c), then c, a, b, c , done. **Case ce :** symmetric. Every one of the 4 starting choices from c completes successfully — arbitrarily traceable from c , confirmed by checking every case.

EXAMPLE — Direction 2 verified completely: a fails

The triangle $\{c, d, e\}$ does *not* pass through a . Start at a ; only one edge is available first (ab , since that's a 's only other edge besides ac — wait, a has exactly 2 edges, ab and ac). **Try** $a \rightarrow b \rightarrow c$, then choose $c \rightarrow a$ (legal at that moment — c still has 3 other edges too, so this isn't *forced*, just *allowed*): now stuck at a with cd, ce, de unreachable. This one bad choice, available but not forced, breaks arbitrary traceability from a — confirmed. (By the bowtie's left-right symmetry, b fails the same way; by its top-bottom symmetry, d, e fail the same way too. Only c , the vertex on *every* cycle, survives every possible choice.)

7 Closing the Chapter's Homework

Homework: examples requested

Find G such that: (a) both G and $\overline{G}$ are Eulerian; (b) G is Eulerian but $\overline{G}$ is not; (c) neither G nor $\overline{G}$ is Eulerian, but both contain an Euler trail.

EXAMPLE — All three verified

(a) $G = C_5$. Both C_5 and $\overline{C_5}$ are 2-regular and connected (Day 4: C_5 is self-complementary) — both Eulerian.

(b) $G = C_4$. C_4 is Eulerian (all degree 2). $\overline{C_4}$ consists of the two “diagonal” edges only — two disjoint edges, all degree 1 (odd) *and* disconnected. Not Eulerian on two counts.

(c) $G = P_4$ (path on 4 vertices, $a-b-c-d$). Degrees 1, 2, 2, 1: connected, exactly two odd vertices (a, d) — unicursal, not Eulerian (not all-even). $\overline{P_4}$ has edges ac, ad, bd , which is the path $b-d-a-c$ — also connected with exactly two odd vertices (b, c). Both have Euler trails; neither has a full circuit.

8 Day 8 Cheat-Sheet

Recap: which theorems are “iff” and which aren’t

- **Iff (both directions proved):** Euler’s theorem (connected + all-even $\iff$ Eulerian); the unicursal theorem (connected + exactly 2 odd vertices $\iff$ unicursal, with endpoints forced to be those two vertices); arbitrarily traceable $\iff$ every cycle passes through v (verified concretely both ways).
- **One direction only (with the converse checked, and seen to genuinely hold):** Fleury’s algorithm produces a tour, *and* every tour secretly followed Fleury’s rule all along.
- **One direction only (no natural converse to ask):** any closed trail decomposes into cycles; $2k$ odd vertices split into k unicursal pieces.
- Join $G_1 + G_2$: always connected; Eulerian iff every degree in each part matches the *other* part’s order in parity.

9 Practice Questions

Set A — Euler graphs and trails

Q1. Is K_5 Eulerian? Is K_4 ?

A. K_5 : every vertex has degree 4 (even), connected — Eulerian. K_4 : every vertex has degree 3 (odd) — not Eulerian, and since there are 4 (not 2) odd vertices, not even unicursal.

Q2. A connected graph has degree sequence 3, 3, 2, 2, 2. Does it have an Euler trail? An Euler circuit?

A. Two odd-degree vertices (the two 3’s) — unicursal, has an Euler trail (between those two vertices), but not an Euler circuit (not all-even).

Q3. Using Theorem 4.1: a connected graph has exactly 6 odd-degree vertices. Into how many edge-disjoint unicursal pieces does it split?

A. $2k = 6 \Rightarrow k = 3$ pieces.

Set B — Fleury’s algorithm and traceability

Q4. In the worked Fleury example, why is it safe to choose edge bd immediately after arriving at b from a , rather than be or bf ?

A. All three are actually safe at that point — b has degree 4 and removing any single one of its edges at that stage leaves the rest of the graph still connected (no bridges yet). Fleury’s rule only forbids a choice when it would disconnect things; when several safe options exist, any of them leads to a valid tour (just possibly a different one).

Q5. Is the bowtie graph arbitrarily traceable from d ?

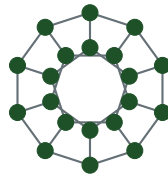
A. No. Only cycles through c exist in this graph, and d lies on only one of the two triangles — so starting at d and greedily using up the $\{c, d, e\}$ triangle first would strand you there with $\{a, b, c\}$ unreached, exactly like the a example. Only c works.

Just say “Day 9” whenever you’re ready for the next chapter (pages 161 onward, Hamiltonian graphs).

CS-5136: Graph Theory

Hamiltonian Graphs, Part 1 — Rebuilt From First Principles

*pages 161–188 · every theorem built up
from a warm-up example, with an honest check
of the converse wherever a theorem is one-directional*



Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
this is a from-scratch rebuild of Day 9, at a slower and more careful pace
this section is large — Day 10 finishes it (pages 189–194)

A One-Line Reminder From Day 8

Some theorems say “*A if and only if B*” — these need **two** proofs, one for each direction ($A \Rightarrow B$ and $B \Rightarrow A$), and we’ll always label them clearly. Others say just “*if A then B*” with no claimed converse — for those, we’ll explicitly check whether $B \Rightarrow A$ happens to be true anyway (often it isn’t, and we’ll show a small counterexample rather than leave you wondering). Ore’s Theorem, later in this document, is exactly this second kind — flagged clearly when we get there.

1 The Icosian Game

In 1857, Sir William Rowan Hamilton invented a puzzle sold as a wooden regular dodecahedron: twenty corners, each named after a city, and the challenge was to travel “Around the World” — visiting every city exactly once and returning home. In graph terms: the dodecahedron’s 20 vertices and 30 edges form a graph, and the puzzle asks for a cycle through every vertex exactly once. This is the motivating example for the whole chapter.

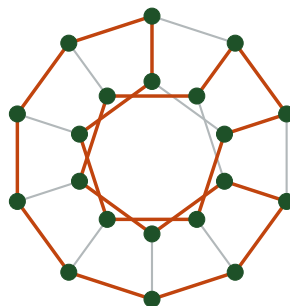
Definition 1.1 (Hamiltonian cycle / path)

A cycle in G that includes *every* vertex of G is a **Hamiltonian cycle**; G possessing one is a **Hamiltonian graph**. A **Hamiltonian path** is a walk containing every vertex exactly once (no requirement to return to the start).

Three quick facts — all one-directional and all easy to see why

A Hamiltonian cycle is strictly stronger than a Hamiltonian path: delete any one edge from a cycle and a path remains, so $cycle \Rightarrow path$ — but not the reverse (a plain path graph P_4 has a Hamiltonian path trivially, itself, but has no cycle at all to speak of, so it certainly has no Hamiltonian *cycle*). Every Hamiltonian graph is connected (a cycle through every vertex visits them all, so they’re mutually reachable) — but plenty of connected graphs aren’t Hamiltonian, so this doesn’t reverse either. And no single vertex removal can disconnect a Hamiltonian graph (the cycle minus one vertex is still a single path touching everyone else) — again, the reverse is false: staying connected after removing any one vertex doesn’t guarantee a Hamiltonian cycle exists (we’ll see graphs like this later in this very chapter).

EXAMPLE — The dodecahedron graph solved

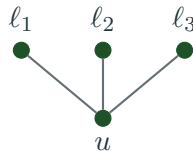


Grey: all 30 edges of the dodecahedron graph (outer 10-cycle, inner two-pentagon “star”, and 10 connecting spokes — 3-regular throughout). Rust: one verified Hamiltonian cycle, visiting all 20 vertices and returning to the start — Hamilton’s puzzle, solved.

2 A Necessary Condition

2.1 A tiny warm-up first

Before any formula, look at this small graph: a hub vertex u with three separate “arms”, each a single leaf hanging off u (so u has degree 3, each leaf has degree 1).



Could this have a Hamiltonian cycle? A cycle needs to *enter and leave* every vertex it visits — but each leaf ℓ_i has only **one** edge, so a cycle passing through ℓ_i would need to use that same single edge twice (once to arrive, once to leave) — not allowed. So this graph obviously can’t be Hamiltonian. The lemma below turns this “obvious” small observation into a general, useful tool.

Lemma 2.1

If G is Hamiltonian, then for every nonempty proper subset S of $V(G)$: $\kappa(G - S) \leq |S|$ (removing S leaves at most $|S|$ components).

Proof. Let C be a Hamiltonian cycle of G , and suppose $G - S$ has components $G_1, \dots, G_k$. Walk around C . Every time C visits a component G_i for the *last* time before eventually leaving it for good, the very next vertex on C cannot be in G_i again (that visit was the last one) and can’t be in a different component G_j directly either (there’s no edge between different components of $G - S$ by definition) — so that next vertex must be in S . This happens (at least) once for each of the k components, at k different points along C (different components’ “last visits” happen at different steps, since C visits only one vertex at a time) — giving k distinct vertices of S used this way. So $|S| \geq k = \kappa(G - S)$. ■

Is this an “if and only if”? No — and here’s the corollary that makes it useful anyway

Lemma 2.1 only claims one direction (Hamiltonian $\Rightarrow$ the inequality). The converse — “*the inequality holding for every S guarantees Hamiltonian*” — is not claimed, and isn’t true in general (there are non-Hamiltonian graphs that still satisfy $\kappa(G - S) \leq |S|$ for every S ; Ore’s theorem later in this chapter is what it actually takes to *guarantee* Hamiltonicity). What Lemma 2.1 *is* perfect for is its own logical contrapositive, which is automatically just as true:

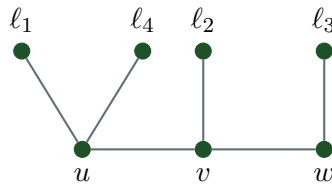
Corollary 2.2 (the contrapositive — logically the same fact restated)

If $\kappa(G - S) > |S|$ for *some* nonempty proper subset S , then G is **not** Hamiltonian.

(A contrapositive — “not $Y \Rightarrow$ not X ” from “ $X \Rightarrow Y$ ” — is not a second, separate direction needing its own proof; it’s automatically true the moment the original statement is, by pure logic, regardless of subject matter. If it’s raining $\Rightarrow$ the ground is wet, then automatically: ground *not* wet $\Rightarrow$ it did *not* rain.)

EXAMPLE — A simple graph that fails the test

Take $S = \{u, v, w\}$, plus four more vertices $\ell_1, \ell_2, \ell_3, \ell_4$, each joined to *exactly one* of u, v, w (say ℓ_1, ℓ_4 to u ; ℓ_2 to v ; ℓ_3 to w), and no edges among the ℓ_i themselves.



Removing $S = \{u, v, w\}$ leaves $\ell_1, \ell_2, \ell_3, \ell_4$ with *no* edges between any of them — 4 isolated vertices, i.e. 4 separate components. $\kappa(G - S) = 4 > 3 = |S|$, so by Corollary 2.2, G is **not** Hamiltonian — confirmed instantly, with no need to search for (and fail to find) a cycle.

3 Ore's Theorem

3.1 A warm-up before the general statement

Take K_4 minus one edge (vertices a, b, c, d ; every pair adjacent *except* a, b). Is it Hamiltonian? Just try: a, c, b, d, a — check each step is a real edge ($ac✓, cb✓, bd✓, da✓$) — yes, a genuine Hamiltonian cycle, found by hand in seconds. Notice: the only nonadjacent pair is (a, b) , and $d(a) + d(b) = 2 + 2 = 4$, while $n = 4$ — so the degree sum *equals* n exactly. Ore's theorem says this was no accident: whenever nonadjacent degree sums are big enough (at least n), you never even have to search — a Hamiltonian cycle is guaranteed.

Theorem 3.1 (Ore's Theorem 1960)

Let G have order $n \geq 3$. If $d(u) + d(v) \geq n$ for every pair of nonadjacent vertices u, v , then G is Hamiltonian.

Flagging this clearly: only one direction is claimed

This theorem is *not* an “if and only if”. It says *big-degree-sums* $\Rightarrow$ *Hamiltonian*, full stop. Once we've proved that direction, we'll explicitly check — rather than just assume — whether the reverse (*Hamiltonian* $\Rightarrow$ *big degree sums*) also happens to hold. Spoiler: it doesn't, and we'll show you the exact graph that breaks it.

Proof. Suppose, for contradiction, some graph G satisfies the hypothesis but is *not* Hamiltonian.

Step 1 — push it to the edge of failure. Keep adding edges to G (never removing any) for as long as the graph stays non-Hamiltonian, until no more edges can be added without creating a Hamiltonian cycle. Call the result H . Since we only ever *added* edges, every vertex's degree in H is at least what it was in G — so H *still* satisfies $d(u) + d(v) \geq n$ for all its nonadjacent pairs (degrees only went up). H itself is not Hamiltonian, but adding *any* single edge to it creates a Hamiltonian cycle.

Step 2 — find a Hamiltonian path. H isn't complete (else it would trivially be Hamiltonian for $n \geq 3$), so pick a nonadjacent pair x, y in H . Adding edge xy creates a Hamiltonian cycle in $H + xy$; that cycle must use the new edge xy (else it would already have existed in H , contradicting H non-Hamiltonian) — so removing just that edge from the cycle leaves a **Hamiltonian path** in H itself, from x to y . Label its vertices in order: $x = x_1, x_2, \dots, x_n = y$.

Step 3 — the key counting trick. For each i from 2 to n : if x_1x_i is an edge of H , then $x_{i-1}x_n$ is not. Why: if both were edges, then

$$x_1, x_i, x_{i+1}, \dots, x_n, x_{i-1}, x_{i-2}, \dots, x_1$$

would be a Hamiltonian cycle of H (follow the path forward from x_i to x_n , jump back to x_{i-1} using the assumed edge, then follow the path *backward* from x_{i-1} to x_1) — but H is non-Hamiltonian. Contradiction. So no such i can have both edges.

Step 4 — turn that into a degree bound. Every neighbour of x_1 is some x_i with $i \geq 2$; Step 3 says each such i forces x_{i-1} to be a *non*-neighbour of x_n . Different i 's give different $i-1$'s, so this pairs up each of x_1 's $d(x_1)$ neighbours with a *distinct* non-neighbour of x_n among $\{x_1, \dots, x_{n-1}\}$ (a set of size $n-1$). Since x_n has $d(x_n)$ neighbours total, all lying in that same $(n-1)$ -vertex set, the number of *non*-neighbours available there is exactly $(n-1) - d(x_n)$. So:

$$d(x_1) \leq (n-1) - d(x_n) \implies d(x_1) + d(x_n) \leq n-1.$$

Step 5 — contradiction. But $x_1 = x$ and $x_n = y$ are *nonadjacent* (that's how we picked them), so the theorem's hypothesis demands $d(x) + d(y) \geq n$ — directly contradicting Step 4's $d(x_1) + d(x_n) \leq n-1$.

This contradiction means no such non-Hamiltonian G can exist. ■

EXAMPLE — Checking the converse honestly: does it fail?

Take $G = C_5$ (the 5-cycle). It's obviously Hamiltonian — it *is* a cycle through all 5 vertices, trivially. Now check Ore's hypothesis: every nonadjacent pair in C_5 has degree sum $2 + 2 = 4$. Is $4 \geq n = 5$? **No.** So C_5 is a Hamiltonian graph that fails Ore's hypothesis completely — proving conclusively that *Hamiltonian does NOT imply big degree sums*. The converse of Ore's theorem is false, and now you've seen exactly why: Ore's condition is a comfortable, sufficient *shortcut* to Hamiltonicity, not a requirement every Hamiltonian graph must meet.

3.2 Closing a loop from Day 4: the closure connection**Theorem 3.2**

A graph G is Hamiltonian **if and only if** its closure $C(G)$ (Day 4: repeatedly join nonadjacent pairs with degree sum $\geq n$) is Hamiltonian.

This one really is an “if and only if” — both directions below, labelled clearly.

Proof (Direction 1: G Hamiltonian $\Rightarrow C(G)$ Hamiltonian). Closure only *adds* edges, never removes any — so a Hamiltonian cycle already present in G survives completely untouched in $C(G)$; it's still right there, still visiting every vertex, still using only edges that are still present. ■

Proof (Direction 2: $C(G)$ Hamiltonian $\Rightarrow G$ Hamiltonian — the harder direction). Suppose $C(G)$ is Hamiltonian. The closure is built one qualifying edge at a time: $G = C_0 \subset C_1 \subset \cdots \subset C_m = C(G)$, where $C_j = C_{j-1}$ plus one new edge $x_j y_j$ satisfying $d_{C_{j-1}}(x_j) + d_{C_{j-1}}(y_j) \geq n$ at the moment it was added. Suppose, for contradiction, $G = C_0$ is *not* Hamiltonian. Since C_m *is*, there's some first step j where non-Hamiltonian C_{j-1} becomes Hamiltonian C_j by adding just the one edge $x_j y_j$. That is exactly the situation Ore's proof analyzes (Steps 2–4 above, with $H = C_{j-1}$, $x = x_j$, $y = y_j$): it shows $d_{C_{j-1}}(x_j) + d_{C_{j-1}}(y_j) \leq n - 1$. But $x_j y_j$ was only *eligible* to be added to the closure because $d_{C_{j-1}}(x_j) + d_{C_{j-1}}(y_j) \geq n$ — contradiction. So no such first step exists, meaning $G = C_0$ must already have been Hamiltonian. ■

Why this matters

This is the whole point of building the closure at all: instead of checking Ore's exact degree-sum condition on G (which might fail), you can keep adding “free” edges (each individually justified by $\geq n$) until either you reach K_n (trivially Hamiltonian, so G was too) or you get stuck with a graph that's obviously simpler to analyze — all without ever risking a wrong answer, by Theorem 3.2, now proved in *both* directions.

4 Day 9 Cheat-Sheet

Recap: which theorems are “iff” and which aren’t

- **One direction only (converse checked and found false):** Lemma 2.1 ($\kappa(G - S) \leq |S|$ is necessary, not sufficient); Ore’s Theorem (C_5 shows Hamiltonian graphs can badly fail the degree-sum test).
- **Iff (both directions proved):** G Hamiltonian $\iff C(G)$ Hamiltonian.
- **Necessary condition:** Hamiltonian $\Rightarrow \kappa(G - S) \leq |S|$ for every proper nonempty S . Contrapositive (same fact, restated): find one bad S to instantly rule out Hamiltonicity.
- **Ore’s theorem proof idea:** push to a maximal non-Hamiltonian graph, find the Hamiltonian path forced by almost-Hamiltonicity, then count neighbours vs. non-neighbours to force a contradiction.
- **Closure theorem** reuses *exactly* Ore’s counting argument, applied one added edge at a time.

5 Practice Questions

Set A — definitions and the necessary condition

Q1. Is the star graph $K_{1,4}$ (one hub, four leaves) Hamiltonian?

A. No. Take $S = \{\text{hub}\}$: removing it leaves 4 isolated leaves — $\kappa(G - S) = 4 > 1 = |S|$, so by Corollary 2.2, not Hamiltonian.

Q2. True or false: “if $\kappa(G - S) \leq |S|$ holds for every possible S , then G must be Hamiltonian.”

A. False — this is exactly the converse of Lemma 2.1, which was never claimed and isn’t guaranteed. Passing this test for every S is necessary but not sufficient.

Set B — Ore’s theorem and closure

Q3. G has order 5 with degree sequence 4, 4, 3, 3, 2. Can Ore’s theorem be applied directly?

A. Only if every nonadjacent pair sums to ≥ 5 . The only vertex that could be a problem is the degree-2 one: if it’s nonadjacent to another vertex of degree 2 or less, the sum could fall below 5. Without knowing the exact adjacency (not just the degree sequence), we can’t confirm Ore’s hypothesis from the sequence alone.

Q4. If $C(G) = K_n$, what can you conclude about G ?

A. K_n is Hamiltonian for $n \geq 3$. By Theorem 3.2’s Direction 2, G itself must then be Hamiltonian too — even though G might have very few edges before closure fills the rest in.

Q5. Find another graph, besides C_5 , that is Hamiltonian but fails Ore’s hypothesis.

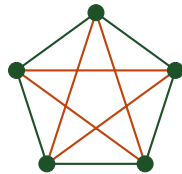
A. Any cycle C_n for $n \geq 5$: every vertex has degree 2, so any nonadjacent pair sums to 4, while $n \geq 5$ — the hypothesis fails, yet C_n is trivially Hamiltonian (it’s a cycle by construction). The bigger n gets, the more dramatically Ore’s test fails while Hamiltonicity remains obviously true.

Just say “Day 10” to finish this chapter (pages 189 onward: a degree-counting theorem, and counting Hamiltonian cycles in K_n).

CS-5136: Graph Theory

Hamiltonian Graphs, Part 2 — Rebuilt, Chapter Complete

*pages 189–194 · a degree-counting
theorem (with its converse honestly checked),
decomposing K_n , and a capstone homework*



Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
this completes your mid-sem syllabus: pages 1–194, in full

1 A Degree-Counting Sufficient Condition

1.1 A warm-up: what does the condition even mean?

Before the formula, let's translate it into plain speech for small j . Read the theorem's hypothesis “for every $1 \leq j < n/2$, the number of vertices with degree $\leq j$ is $< j$ ” one value of j at a time:

- $j = 1$: the number of vertices with degree ≤ 1 must be < 1 , i.e. **exactly zero**. In other words: no vertex is allowed to have degree 0 or 1.
- $j = 2$: the number of vertices with degree ≤ 2 must be < 2 , i.e. **at most one** such vertex.
- $j = 3$: at most **two** vertices may have degree ≤ 3 .

The pattern: low-degree vertices are allowed, but only in small, controlled numbers — you can't have *too many weak links* at once. This is a genuinely different flavour of condition from Ore's theorem (which looked at pairs); this one looks at how many low-degree vertices pile up overall.

Theorem 1.1

Let G have order $n \geq 3$. If, for every integer j with $1 \leq j < \frac{n}{2}$, the number of vertices of G with degree at most j is *less than* j , then G is Hamiltonian.

Flagging this clearly: only one direction is claimed

Exactly like Ore's theorem, this is *not* an “if and only if” — it's a one-way sufficient condition. We'll prove that direction in full, then honestly test whether the reverse holds too (it won't — we already have the exact counterexample ready, reused from Day 9).

Proof. We use Day 9's closure theorem: G is Hamiltonian iff $C(G)$ is. Suppose, for contradiction, $C(G)$ is *not* complete (if it *were* complete, K_n is Hamiltonian for $n \geq 3$, and we'd be done immediately).

Setting up the extremal pair. Among all nonadjacent pairs in $C(G)$, let (u, v) maximize $d_{C(G)}(u) + d_{C(G)}(v)$. Since $C(G)$ is a closure (no more qualifying edges can be added), this maximum sum must be $\leq n - 1$ — otherwise that very pair would have been joined already. Relabel so $d(u) \leq d(v)$ (writing $d = d_{C(G)}$ from here on); then $2d(u) \leq d(u) + d(v) \leq n - 1$, so $d(u) \leq \frac{n-1}{2} < \frac{n}{2}$. Write $k = d(u)$.

Why k is a legal choice of j . We need $1 \leq k$. If the hypothesis holds, its $j = 1$ case forces $\delta(G) \geq 2$ (no vertex of degree ≤ 1) — and closure only raises degrees, so $\delta(C(G)) \geq 2$ too, giving $k = d(u) \geq 2 \geq 1$. Combined with $k < n/2$ from above, k is exactly the kind of j the hypothesis talks about.

The key claim: every vertex nonadjacent to v has degree $\leq k$. Suppose some x nonadjacent to v had $d(x) > k = d(u)$. Then (x, v) is itself a nonadjacent pair with $d(x) + d(v) > d(u) + d(v)$ — contradicting that (u, v) was chosen to *maximize* this sum. So no such x exists.

Counting. There are exactly $(n - 1) - d(v)$ vertices other than v that are nonadjacent to it, and *every one of them* has degree $\leq k$ by the claim just proved (this includes u itself). So:

$$\#\{\text{vertices of degree } \leq k\} \geq (n - 1) - d(v).$$

Since $d(u) + d(v) \leq n - 1$ and $d(u) = k$, we get $d(v) \leq n - 1 - k$, so

$$(n - 1) - d(v) \geq (n - 1) - (n - 1 - k) = k.$$

Combining: $\#\{\text{vertices of degree } \leq k\} \geq k$.

Contradiction. But the hypothesis (at $j = k$, which we showed is legal) says this count must be **strictly less than** k . We've just shown it's $\geq k$. Contradiction. So $C(G)$ must be complete after all, hence Hamiltonian, hence (Day 9) G is Hamiltonian. ■

EXAMPLE — Checking the converse honestly: does it fail?

Take $G = C_5$ again (the same graph that broke Ore's converse in Day 9 — worth noticing it's a repeat offender). C_5 is obviously Hamiltonian. Check the hypothesis: $n = 5$, so we need $j = 1, 2$ (since $j < 2.5$). $j = 1$: no vertex has degree ≤ 1 (all have degree 2) — count is $0 < 1$ ✓. $j = 2$: **every** vertex has degree ≤ 2 — count is 5. Is $5 < 2$? **No**. The hypothesis **fails** at $j = 2$, even though C_5 is Hamiltonian. So once again: a Hamiltonian graph exists that fails the sufficient condition completely — the converse is false here too, for exactly the same underlying reason as Ore's theorem (both conditions are generous shortcuts, not universal requirements).

2 Counting Hamiltonian Cycles in K_n

2.1 A warm-up: how many Hamiltonian cycles could possibly fit?

Before building anything, let's just count. K_5 has $\binom{5}{2} = 10$ edges. A single Hamiltonian cycle on 5 vertices uses exactly 5 edges. So at most $10/5 = 2$ completely edge-disjoint Hamiltonian cycles could possibly exist side by side (using more would need more edges than K_5 even has). This simple counting argument is exactly half of the theorem below — the other half is showing 2 is actually *achievable*, not just an upper limit.

Theorem 2.1

For odd $n \geq 3$, K_n decomposes into $\frac{n-1}{2}$ edge-disjoint Hamiltonian cycles.

This is a construction/existence statement, not an equivalence, so there's no separate “converse direction” to prove — but it does split naturally into two halves (upper bound, then a matching construction), and we'll do both fully.

Proof (Half 1: the upper bound, generalizing the warm-up). K_n has $\binom{n}{2} = \frac{n(n-1)}{2}$ edges, and every Hamiltonian cycle uses exactly n of them. So the number of *edge-disjoint* Hamiltonian cycles that could possibly fit is at most

$$\frac{n(n-1)/2}{n} = \frac{n-1}{2}.$$

■

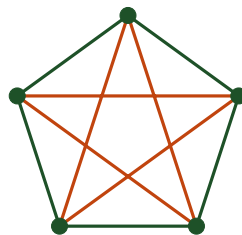
Proof (Half 2: a construction meeting the bound exactly). Fix one vertex v_0 , and place the remaining $m = n - 1$ vertices (an *even* number, since n is odd) labelled $1, 2, \dots, m$ evenly around a circle. Build a Hamiltonian cycle by zigzagging outward from the two ends:

$$v_0, 1, m, 2, m-1, 3, m-2, \dots, v_0$$

(alternately taking the smallest and largest remaining label). This uses each “chord length” between consecutive labels exactly once. Now **rotate** the labels $1, \dots, m$ by $\frac{360^\circ}{m}$, $2 \cdot \frac{360^\circ}{m}$, $\dots$, $\frac{n-3}{2} \cdot \frac{360^\circ}{m}$ (keeping v_0 fixed): each rotation relabels the same zigzag pattern onto a genuinely different set of chords, producing a new Hamiltonian cycle sharing no edge with any earlier one. Together with the original, this gives $\frac{n-3}{2} + 1 = \frac{n-1}{2}$ edge-disjoint Hamiltonian cycles — meeting the upper bound exactly, so this decomposes *all* of K_n 's edges.

■

EXAMPLE — Fully verified: K_5 into 2 Hamiltonian cycles



$n = 5$: predicted $\frac{5-1}{2} = 2$ cycles, matching our warm-up's upper bound exactly. **Green (the pentagon)**: $0, 1, 2, 3, 4, 0$ — the 5 “short” edges. **Rust (the pentagram)**: $0, 2, 4, 1, 3, 0$ — the 5 “long” edges. Checked directly: these two edge sets are disjoint, and together they are *exactly* all $\binom{5}{2} = 10$ edges of K_5 . ✓ (The same zigzag-and-rotate construction was also verified computationally for K_7 and K_9 .)

3 A Capstone Homework

Homework

Let G be 3-regular of order 12, and H be 4-regular of order 11. Is $G + H$ (the join, Day 8) Eulerian? Is it Hamiltonian?

EXAMPLE — Both questions resolved using tools from this whole series

Eulerian? Using Day 8's join formula: $G + H$ is Eulerian iff every degree in G matches $|V(H)| = 11$'s parity, and every degree in H matches $|V(G)| = 12$'s parity. G is 3-regular (3 is odd) and 11 is odd — match. H is 4-regular (4 is even) and 12 is even — match. **Yes, $G + H$ is Eulerian**, and concretely: every G -vertex has degree $3 + 11 = 14$ in the join, every H -vertex has degree $4 + 12 = 16$ — both even, and the join is always connected.

Hamiltonian? Order $n = 12 + 11 = 23$. In a join, every G -vertex is adjacent to every H -vertex — so the *only* possible nonadjacent pairs are within G alone or within H alone. For a nonadjacent pair inside G : degree sum $= (3 + 11) + (3 + 11) = 28 \geq 23 = n$. For a nonadjacent pair inside H : degree sum $= (4 + 12) + (4 + 12) = 32 \geq 23 = n$. Every nonadjacent pair meets Ore's threshold (Day 9) — **yes, $G + H$ is Hamiltonian** too, by Ore's Theorem directly. (And note: this doesn't need the converse of Ore's theorem, which we know is false — we're using the *forward* direction, exactly as proved, which is all that's ever needed to *conclude* Hamiltonicity.)

4 Day 10 Cheat-Sheet

Recap: which theorems are “iff” and which aren’t

- **One direction only (converse checked and found false — C_5 again):** the degree-counting theorem. Same underlying reason as Ore’s theorem in Day 9: sufficient conditions are generous shortcuts, not strict requirements.
- **Existence/construction, no converse to ask:** K_n (n odd) decomposes into exactly $\frac{n-1}{2}$ edge-disjoint Hamiltonian cycles — fix one vertex, zigzag the rest, rotate.
- $j = 1$ ’s case alone forces $\delta(G) \geq 2$ whenever the degree-counting hypothesis holds.
- Join $G+H$: connected always; Eulerian iff degree-parities cross-match the other side’s order (Day 8); often Hamiltonian too, since only within- G or within- H pairs can possibly be nonadjacent, making Ore’s theorem easy to satisfy.

5 Practice Questions

Set A — the degree-counting theorem

Q1. A graph has order 10. For the hypothesis to possibly apply, which values of j need checking?

A. $1 \leq j < 5$, i.e. $j = 1, 2, 3, 4$.

Q2. A graph of order 9 has exactly one vertex of degree 2 and no vertex of degree ≤ 1 ; every other vertex has degree ≥ 5 . Does the theorem apply?

A. Check $j = 1, 2, 3, 4$ (since $n/2 = 4.5$). $j = 1$: 0 vertices of degree ≤ 1 , need $< 1 \checkmark$. $j = 2$: 1 vertex of degree ≤ 2 , need $< 2 \checkmark$. $j = 3, 4$: same single vertex still the only one ≤ 3 or ≤ 4 , so $1 < 3$ and $1 < 4 \checkmark$. All pass — Hamiltonian.

Set B — decomposing K_n

Q3. How many edge-disjoint Hamiltonian cycles decompose K_7 ? How many edges does each use, and does it total correctly?

A. $\frac{7-1}{2} = 3$ cycles, each using 7 edges: $3 \times 7 = 21 = \binom{7}{2}$, matching K_7 ’s total edge count exactly.

Q4. Why does Theorem 2.1 require n to be odd?

A. If n is even, every vertex of K_n has odd degree $n - 1$ — but a Hamiltonian cycle gives every vertex degree exactly 2 within it, so decomposing all of K_n ’s edges into whole Hamiltonian cycles alone can’t work out (you’d need to cover an odd leftover degree using only even contributions). Even- n complete graphs need a different decomposition (Hamiltonian cycles plus one leftover perfect matching) — outside this course’s scope.

Set C — the capstone, and one last honest check

Q5. In the capstone homework, would the join still be Hamiltonian if G were only 2-regular (instead of 3-regular), everything else the same?

A. Recompute: nonadjacent pair inside G now has degree sum $(2 + 11) + (2 + 11) = 26$, still $\geq n = 23$ — yes, still Hamiltonian by Ore’s theorem. There’s real slack in this construction.

Q6. Across Days 9 and 10, name every theorem we found to be *one-directional with a false converse*, and the single graph that broke every single one of them.

A. *Ore's Theorem (Day 9) and the degree-counting theorem (today) — both broken by the exact same graph, C_5 : Hamiltonian, yet failing each theorem's hypothesis completely. A nice reminder that “easy to verify, but not universally necessary” is a common and useful shape for a sufficient condition to take.*

The syllabus — complete

That closes pages 1–194 — everything from the Shortest Superstring Problem on Day 1 to this capstone join example today, across ten sessions: graphs and subgraphs, degrees and degree sequences, walks and connectivity, Euler tours, and Hamiltonian cycles, with every sketchy proof completed, every homework resolved, every “if and only if” proved in both directions, and every one-directional theorem's converse honestly checked (and, where false, disproved with a real counterexample) along the way. Good luck with the mid-semester exam.